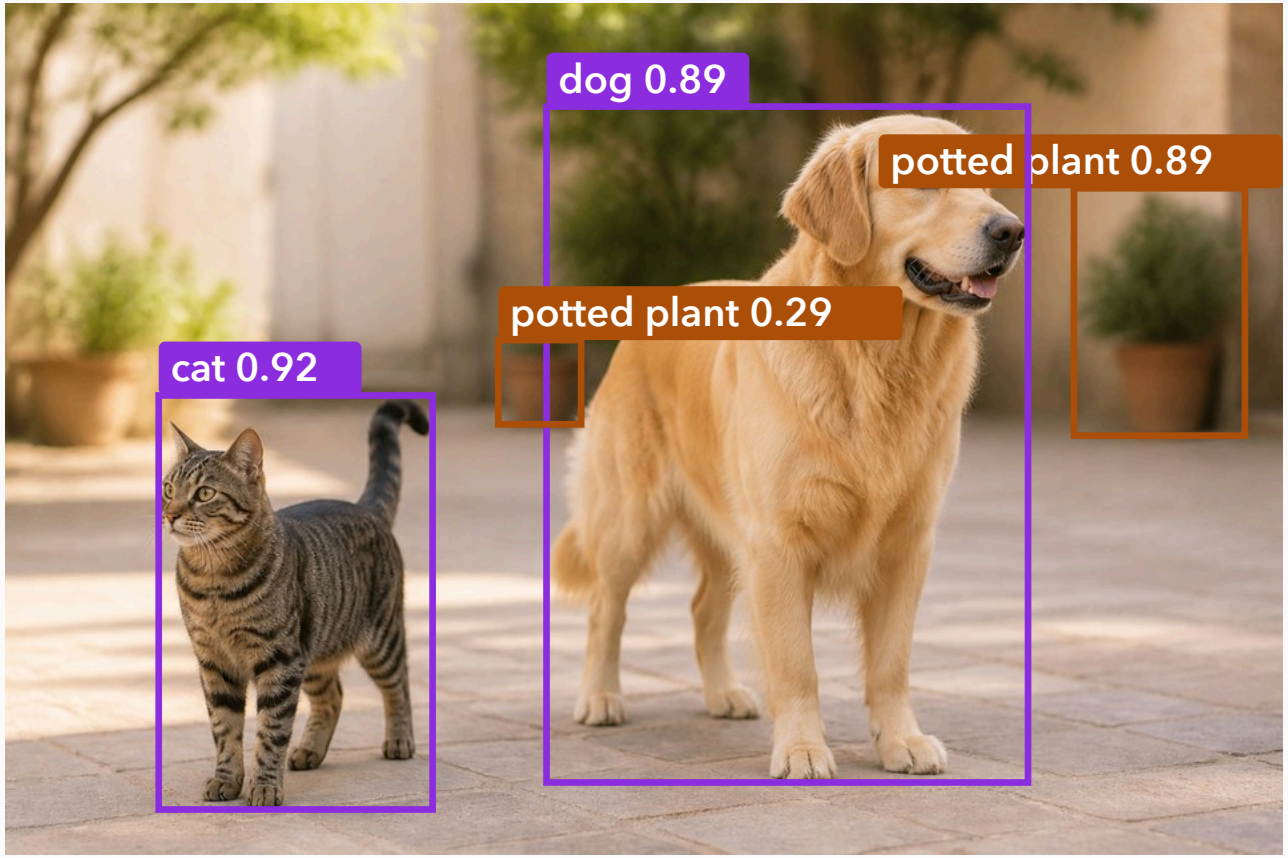
# Object detection: from one label to a set of objects

A detector looks at a photo and returns a list: one entry per object, each a class and a box. How can a network with a fixed-size output return a list whose length changes from photo to photo?

ES 667 Deep Learning · Lecture 22
Nipun Batra · IIT Gandhinagar




> The real output of a small detector, YOLOv8n, on today's photo: four objects, each with a class, a score and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

# What is in this photo?



| ImageNet class | probability |
|---|---|
| golden retriever | 0.192 |
| flat-coated retriever | 0.015 |
| tiger cat | 0.013 |
| Labrador retriever | 0.012 |
| tabby | 0.010 |

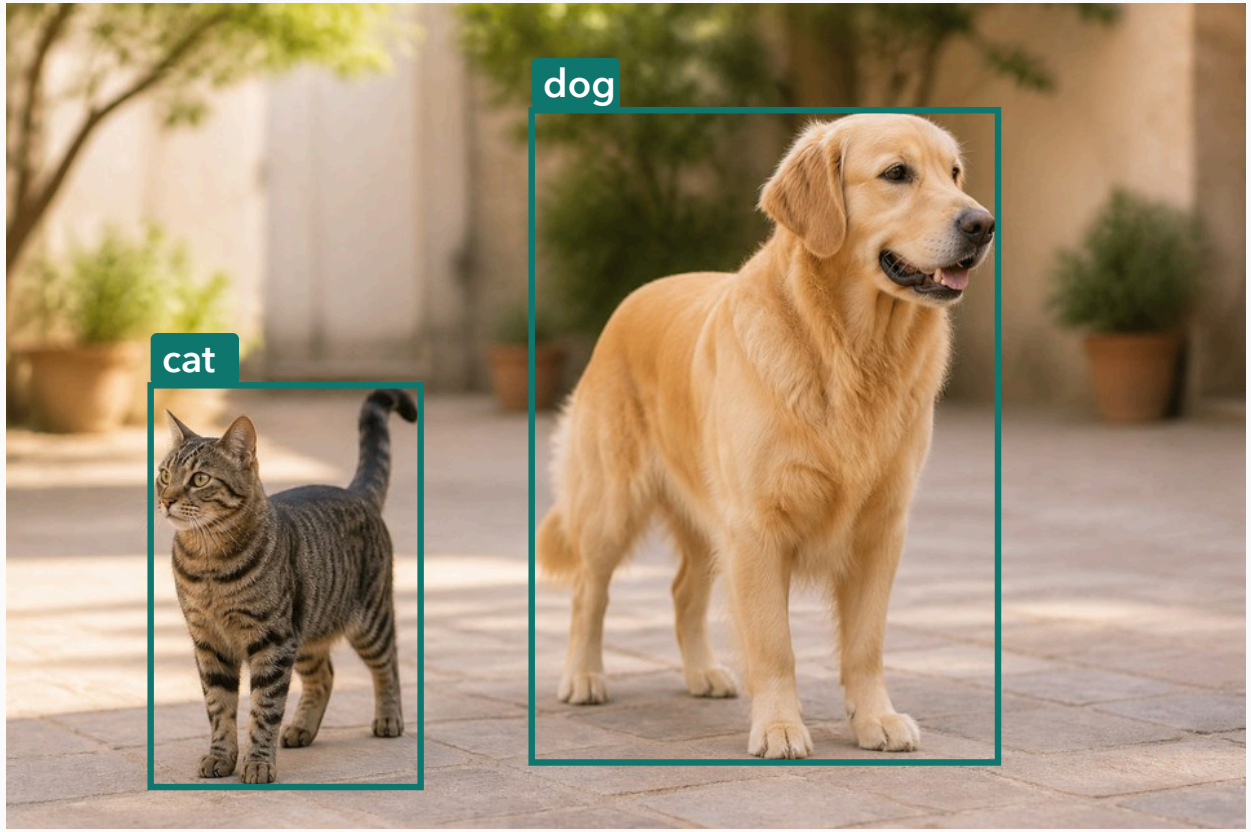
One label for the whole photo. Its 118 dog classes together hold 0.39; the five cat classes, 0.03.

> The cat appears only as a weak runner-up, tiger cat at 0.013. The answer cannot say “a dog and a cat”, how many, or where.

ResNet-50 with torchvision’s IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 1

# What we want: a list, one entry per object




| class | box: left, top, right, bottom (pixels) |
|---|---|
| dog | (407, 83, 768, 588) |
| cat | (112, 296, 321, 607) |

The boxes are our annotation of this photo, in pixels from its top-left corner.

> Second entry: cat. Two objects, two entries. A photo with five objects gets five.

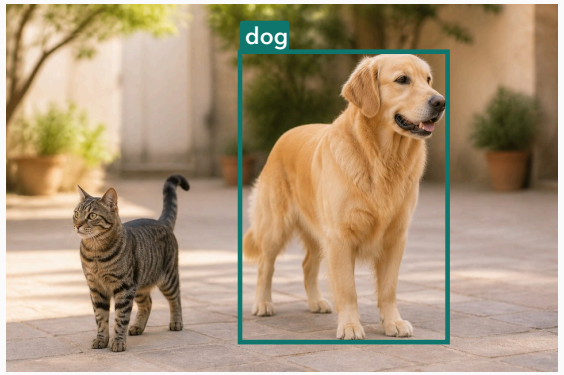
The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

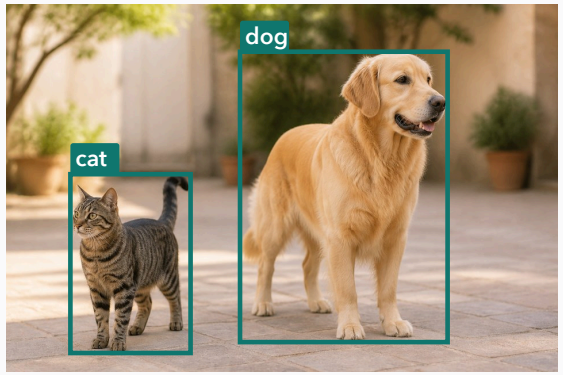
# Four tasks, four kinds of answer



Classification: one label




Localisation: one label, one box




Detection: a class and a box per object



Segmentation: which pixels (Lecture 23)

> Segmentation: which pixels belong to each object. The masks here are from a real YOLOv8n-seg run; this is next lecture.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

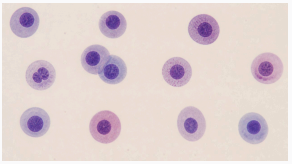
▶ Notebook, part 1

# A street in one glance: seven objects, four classes

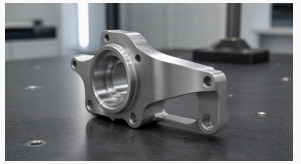





parcels on a belt



cells on a slide



parts to inspect

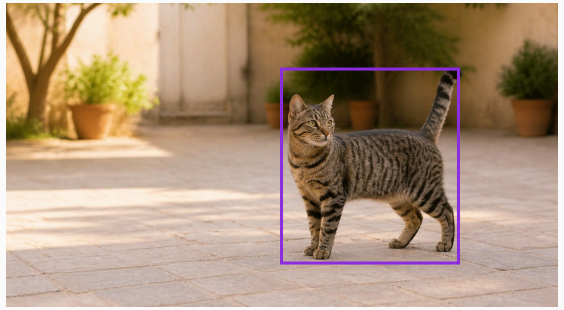
Same task, other classes: count parcels, count cells, find defects.

> The same task with other classes: parcels on a conveyor, cells under a microscope, defects on a machined part.

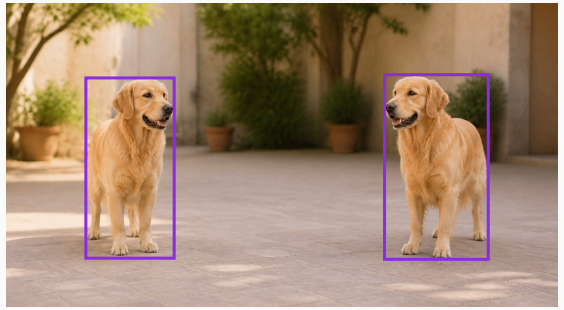
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

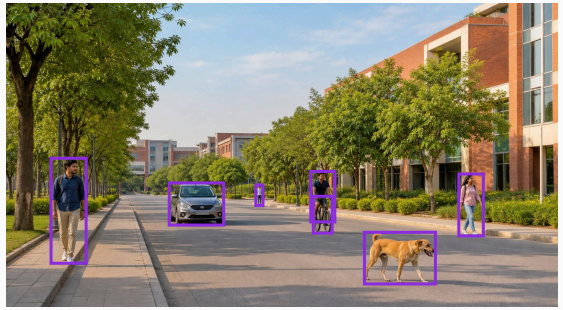
# The number of objects changes from photo to photo



**1** · one cat



**2** · two dogs



**7** · a street



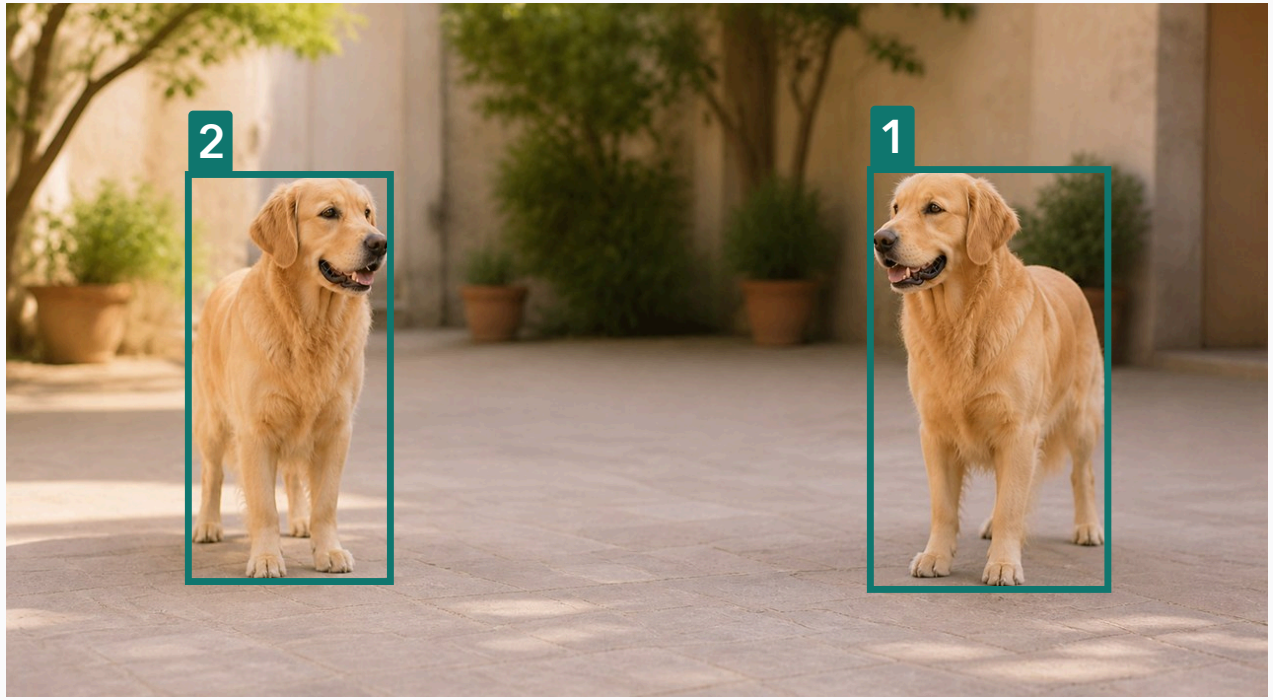
**11** · a path of people

A busy path: eleven people. The length of the answer depends on the photo.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

# A list with no order: a set




$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

Number the dogs either way: it is the same answer. Only each class stays paired with its own box.

> Or right one first. Both lists describe the same scene: the answer is a set of N (class, box) pairs, N varies, and order means nothing.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

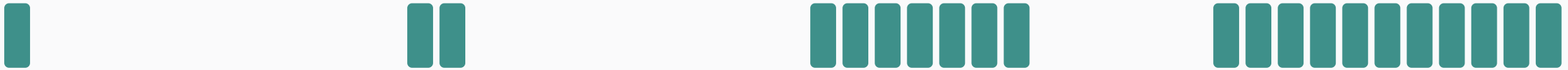
▶ Notebook, part 1

# A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

Today's question: **how can a fixed-size output describe a set of any size, in no order?**

> That is the question of the lecture.

▶ Notebook, part 1

# Today’s route: two answers to one question

## A · a candidate at every place

- Each cell of a grid writes one record
- Train the cells near each object
- Keep the best record per object (NMS)

YOLO-style dense detectors · Parts 4–6

## B · a fixed set of slots, matched

- 100 slots, each may say “no object”
- Match slots to objects before the loss
- No clean-up needed

DETR · Part 7

> Answer B: a fixed set of 100 slots; before computing the loss, match slots to objects. This is DETR. We end by measuring both.

▶ Notebook, part 2

# Part 2 · Describe a box, compare two boxes



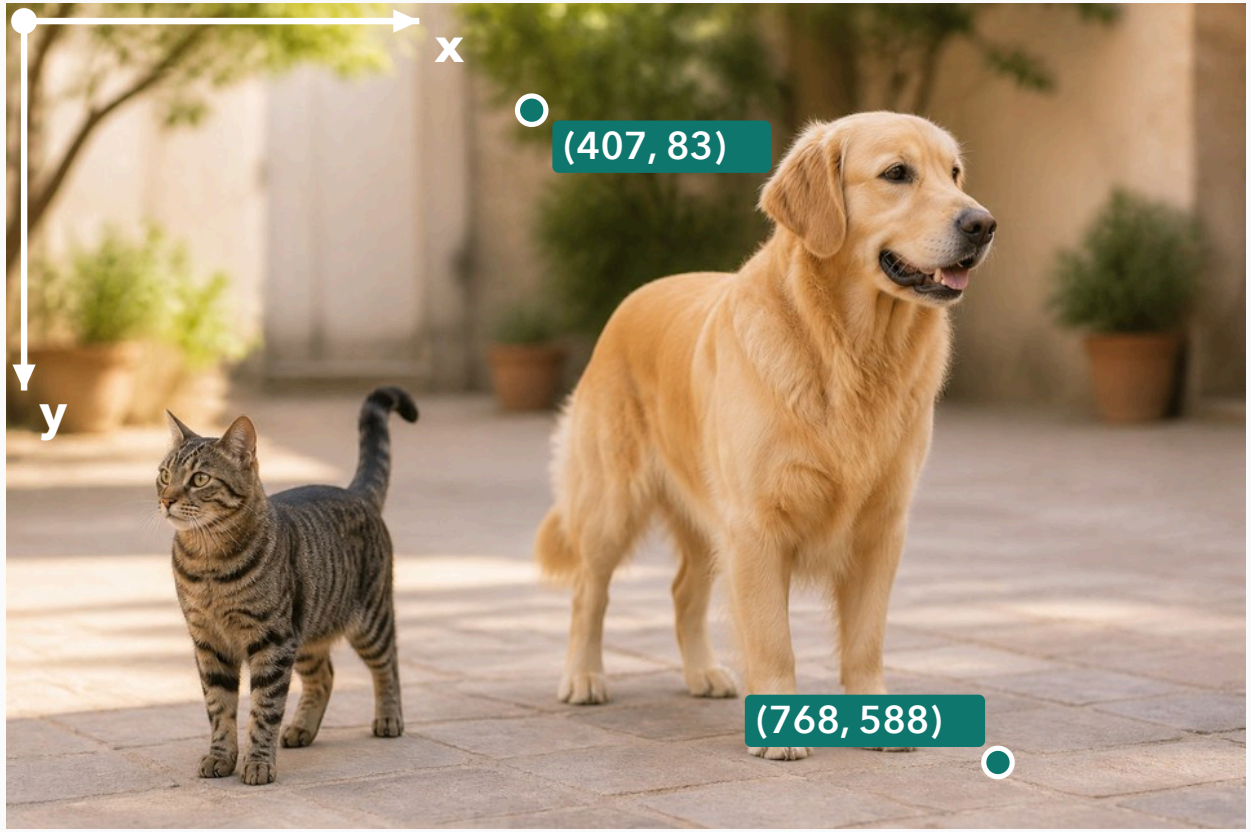
Every later step needs two small tools.

1. Write a box as four numbers, in two common formats.
2. Measure how well two boxes agree: intersection over union, IoU, computed by hand.
3. Try IoU live, then write it in six lines of PyTorch.

> Two tools: four numbers for a box, and IoU to compare two boxes.

▶ Notebook, part 2

# Positions in a photo: pixels from the top-left corner




The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

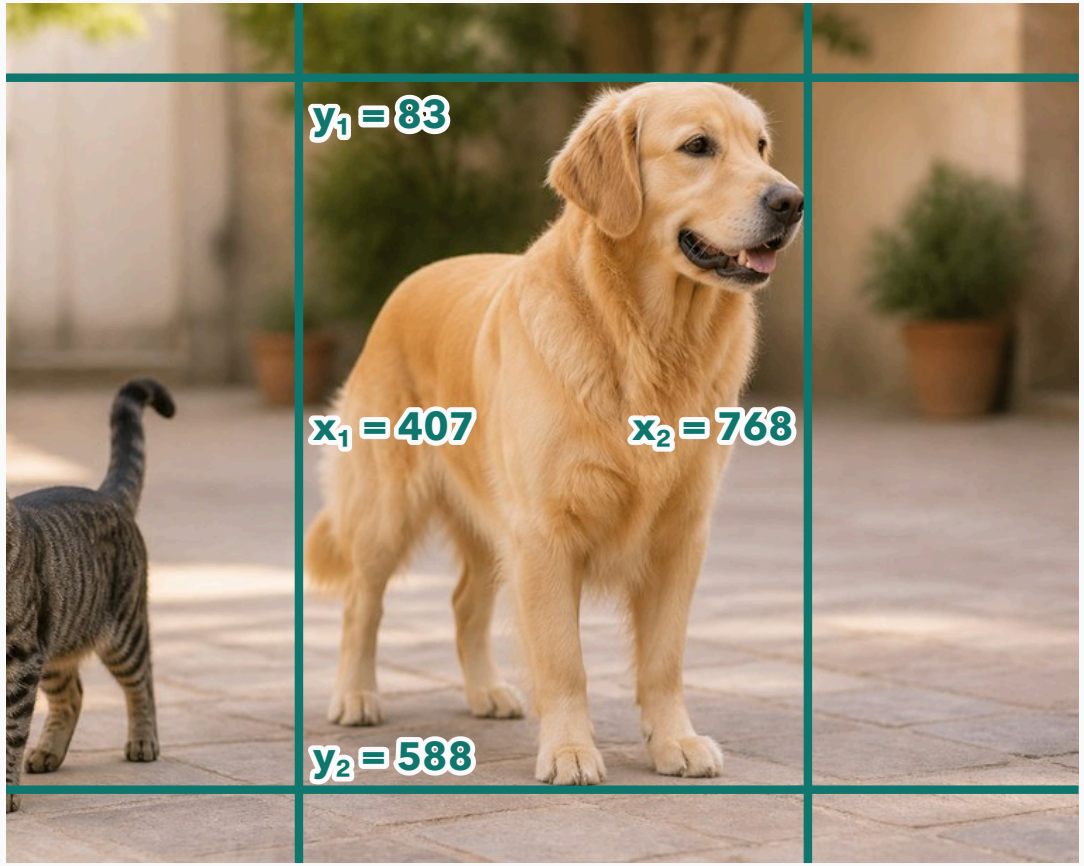
Two corners of the dog: top-left (407, 83), bottom-right (768, 588).

> The dog's top-left corner is at (407, 83); its bottom-right corner at (768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

# A box is four numbers: its left, top, right and bottom edges




Corner format:

$$(x_1,\ y_1,\ x_2,\ y_2)$$

$$= (407,\ 83,\ 768,\ 588)$$

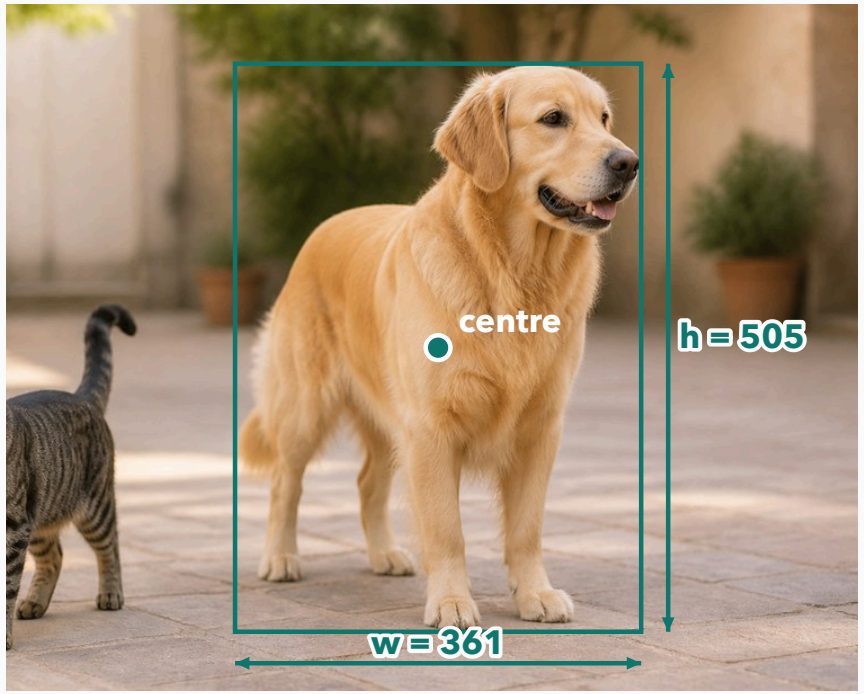
($x_1$, $y_1$) is the top-left corner, ($x_2$, $y_2$) the bottom-right. torchvision uses this format.

> Bottom edge: y at most 588, the paws. Four numbers: (407, 83, 768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

# The same box as a centre and a size




$$w = x_2 - x_1 = 768 - 407 = 361$$

$$h = y_2 - y_1 = 588 - 83 = 505$$

$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

$$(c_x, c_y, w, h) = (587.5,\ 335.5,\ 361,\ 505)$$

> Same box, other format: centre (587.5, 335.5), size 361 × 505. Either format can be computed from the other.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

# Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

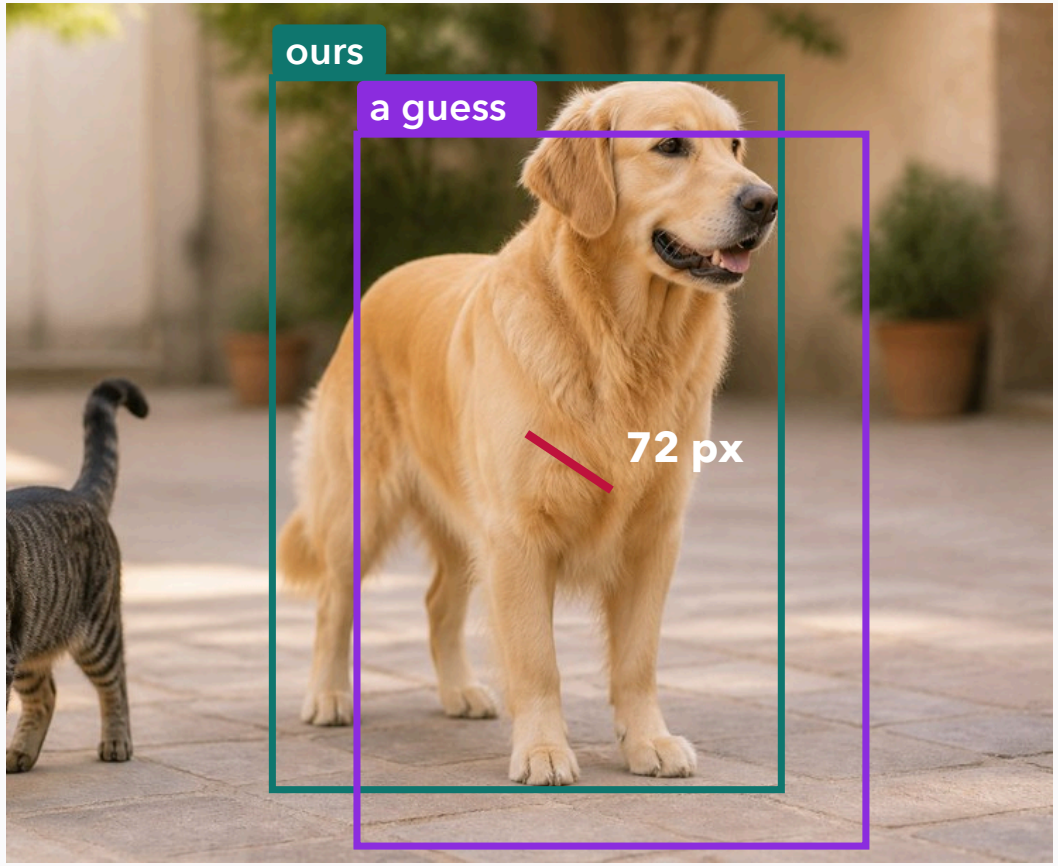
$$= (0.612, 0.524, 0.376, 0.789)$$

Halve the photo to 480 × 320: every pixel number halves, the normalised numbers do not change. The dog's centre is 61% of the way across and 52% of the way down, at any resolution.

> These numbers survive resizing. Networks often predict normalised boxes; DETR, in Part 7, does exactly this.

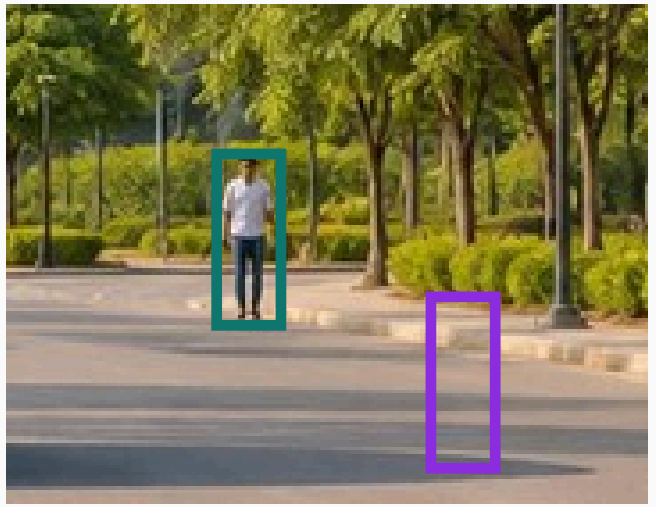
▶ Notebook, part 2

# Two boxes for one dog: how well do they agree?




Idea 1: the distance between centres, $\sqrt{(60^2 + 40^2)} = 72$ px.

The same 72 px slip on a far-away person 18 px wide:



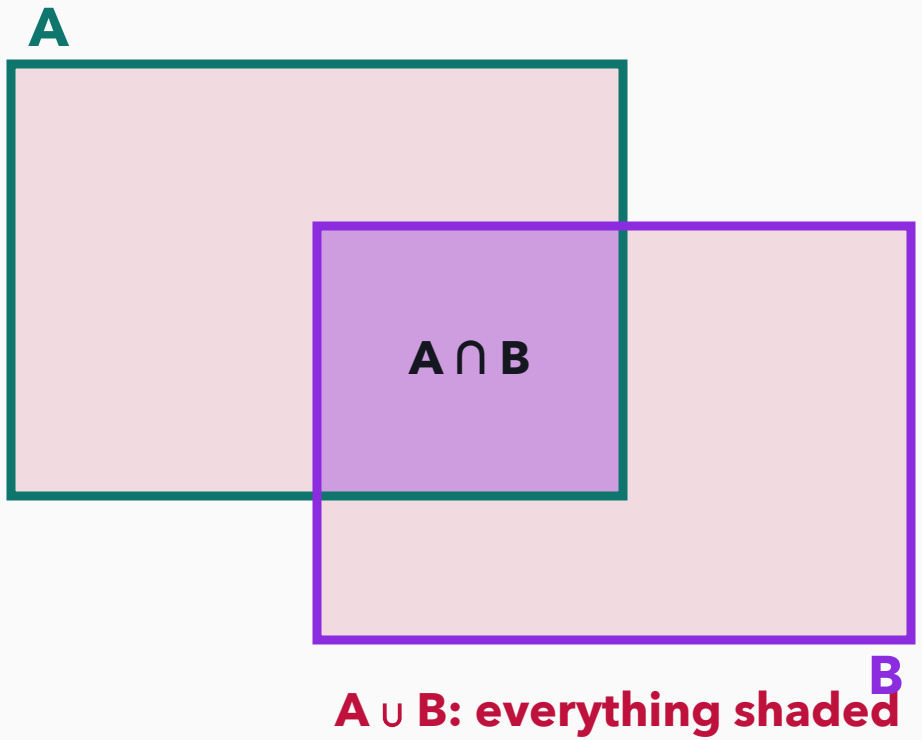
A slip that is fine for the dog misses the small person completely. We need a score that scales with the box.

> For the dog, 72 px is a small slip. For a person 18 px wide in the street photo, the same slip leaves the boxes apart. Compare overlap, not distance.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

# Intersection over union (IoU)




Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

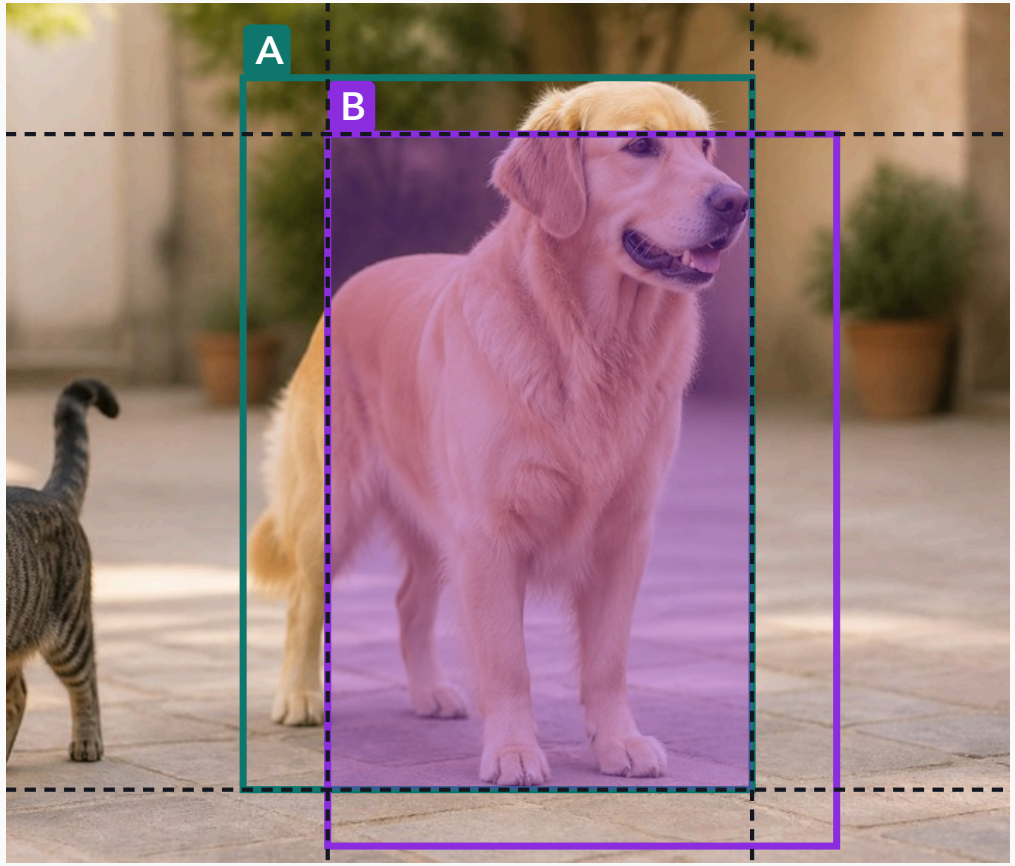
$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

Identical boxes: IoU = 1. Boxes that do not touch: IoU = 0.

> IoU divides the first by the second. It is a fraction of area, so it does not care how big the boxes are.

▶ Notebook, part 2

# IoU, step 1: the overlap rectangle




$$\text{left} = \max(407,\ 467) = 467$$

$$\text{top} = \max(83,\ 123) = 123$$

$$\text{right} = \min(768,\ 828) = 768$$

$$\text{bottom} = \min(588,\ 628) = 588$$

$$\text{area} = 301 \times 465 = 139{,}965$$

> Its area: 301 × 465 = 139,965 square pixels. (If a width came out negative, the boxes would not touch: area 0.)

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

# IoU, step 2: the union, then the ratio

$$\text{area}(A) = 361 \times 505 = 182{,}305, \qquad \text{area}(B) = 182{,}305$$

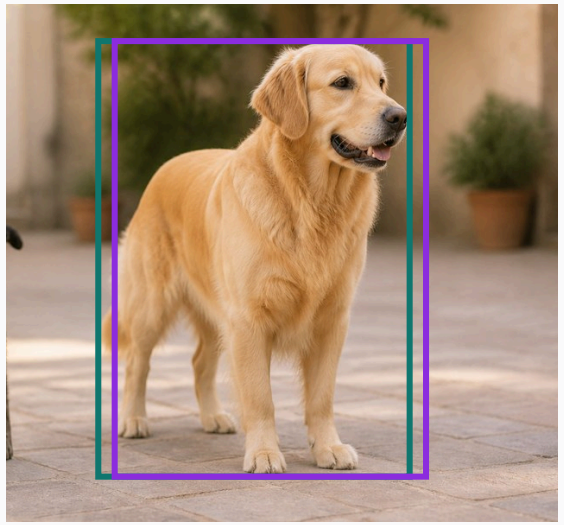
$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

$$\text{IoU} = \frac{139{,}965}{224{,}645} = 0.623$$

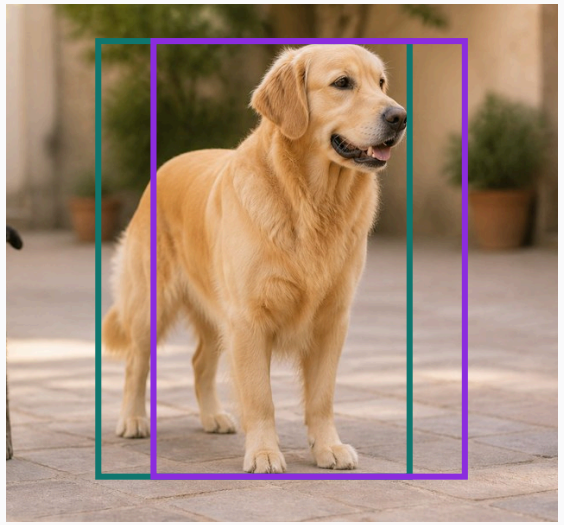
> IoU = 0.623. A move of 60 px right and 40 px down cost more than a third of the score.

▶ Notebook, part 2

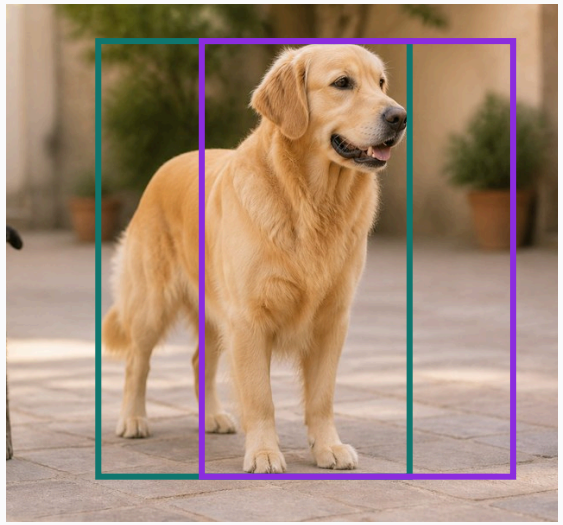
# What IoU values look like



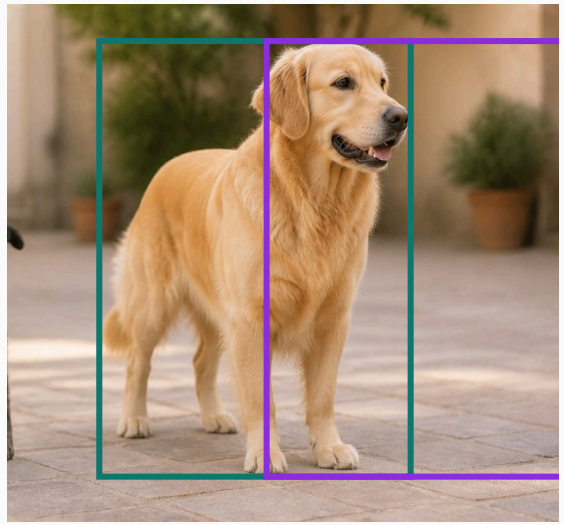
IoU 0.9 · moved 19 px



IoU 0.7 · moved 64 px



IoU 0.5 · moved 120 px



IoU 0.3 · moved 194 px

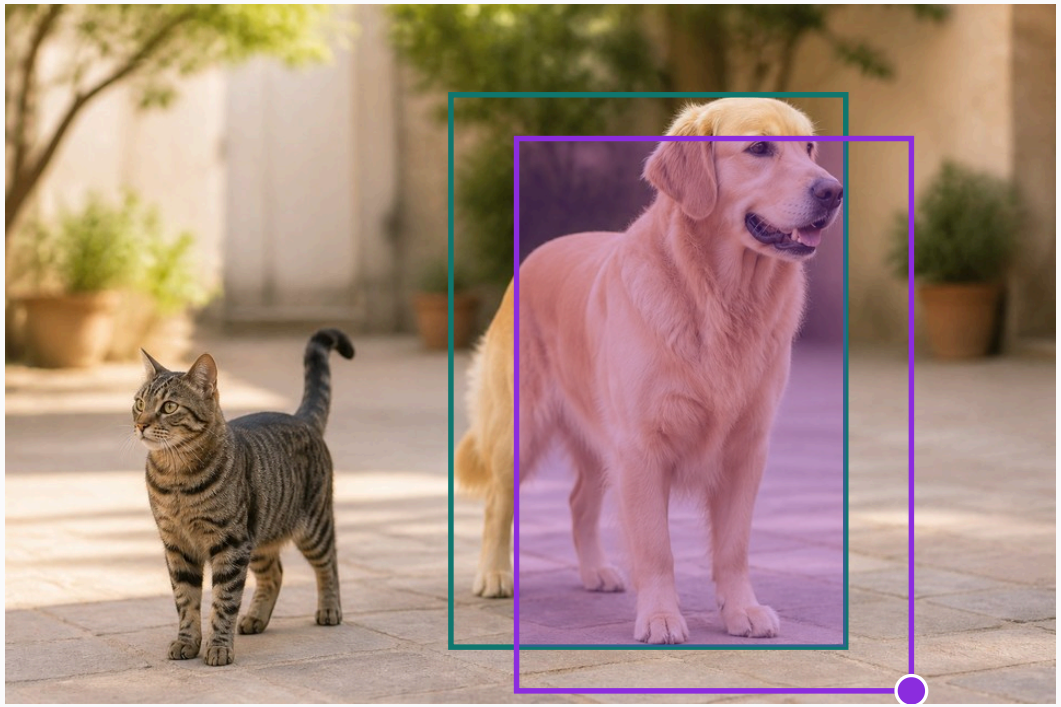
Benchmarks usually count IoU ≥ 0.5 as a hit. The real YOLOv8n box for this dog has IoU 0.983 with ours.

> Most benchmarks call IoU ≥ 0.5 a hit. A trained detector does much better on this dog: 0.983.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

# Try it: drag a box over the dog



| | |
|---|---|
| overlap | 139,965 px² |
| union | 224,645 px² |
| IoU | **0.623** |

> Drag the purple box. Can you find a box with IoU 0.5 that looks good? One with IoU 0.7 that looks bad?

▶ Notebook, part 2

# IoU in code: every pair at once

```python
def box_iou(a, b):                  # a: (N, 4), b: (M, 4), corners
    lt = torch.max(a[:, None, :2], b[None, :, :2])  # overlap's top-left
    rb = torch.min(a[:, None, 2:], b[None, :, 2:])  # overlap's bottom-right
    inter = (rb - lt).clamp(min=0).prod(-1)         # (N, M); 0 if apart
    area = lambda x: (x[:, 2:] - x[:, :2]).prod(-1)
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)
```

```python
box_iou(torch.tensor([[407., 83., 768., 588.]]), torch.tensor([[467., 123., 828., 628.]]))
# tensor([[0.6230]])  -- torchvision.ops.box_iou gives the same
```

> On our two boxes: 0.6230. torchvision.ops.box_iou returns the same N × M matrix.

▶ Notebook, part 3

# Part 3 · One box is easy; several are not



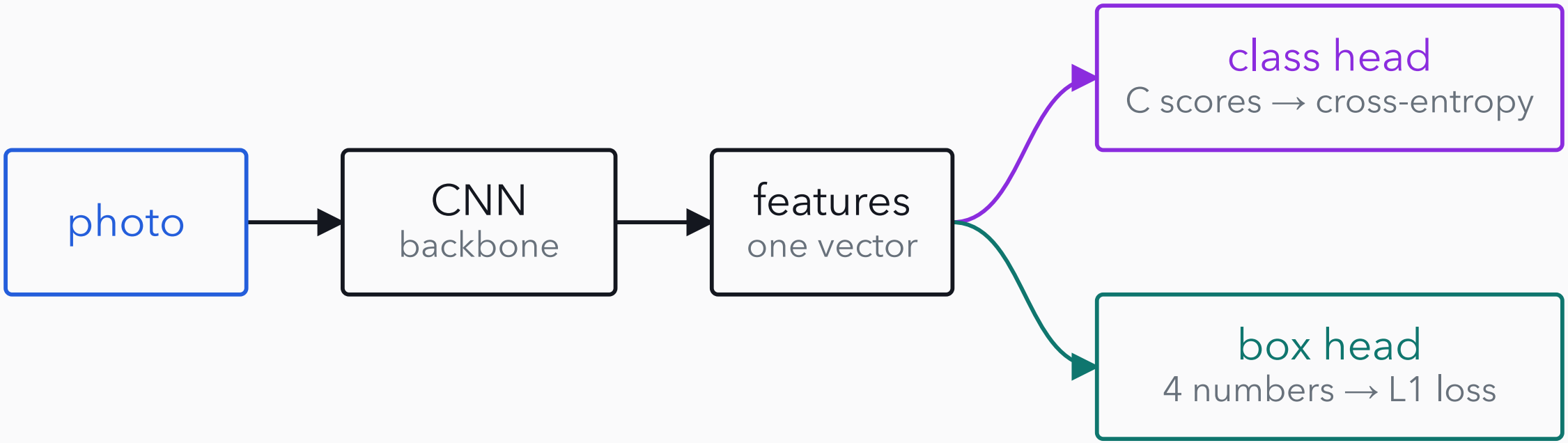
A network can output four numbers for one box. Asking it for several boxes shows the real problem.

1. Add a box head to a classifier and train it with an L1 loss.
2. Ask the same head for two animals, and see it hedge.
3. Give it K slots: count and order break the loss.

> One box is regression; several boxes expose the assignment problem.

▶ Notebook, part 3

# One object: a box head beside the class head




> Train both at once: cross-entropy for the class, and a loss that compares the four box numbers with the annotation's.

▶ Notebook, part 3

# Training the box head: compare four numbers

$$\mathcal{L}_{\text{box}} = \sum_{k=1}^{4} \left| \hat{b}_k - b_k \right|$$

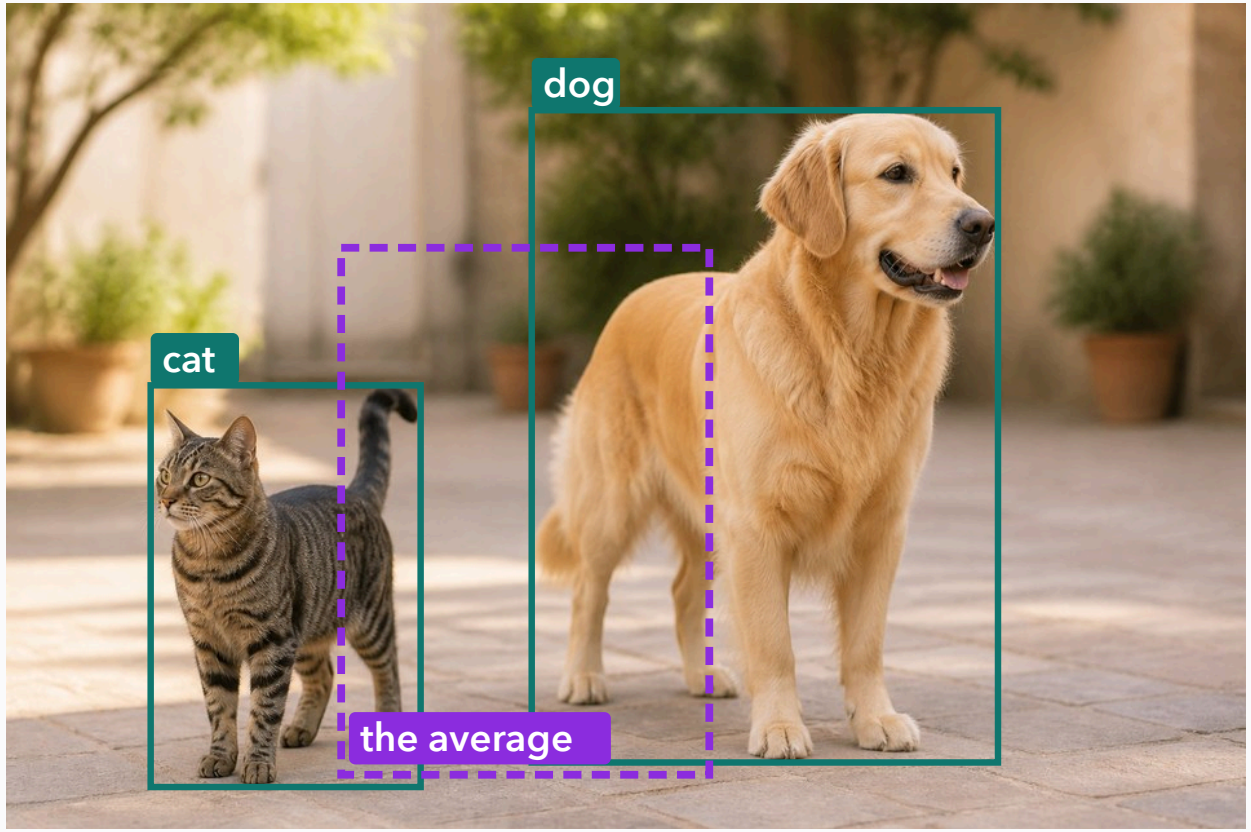
| | $c_x$ | $c_y$ | $w$ | $h$ |
|---|---|---|---|---|
| target (the dog) | 0.612 | 0.524 | 0.376 | 0.789 |
| prediction (the guess) | 0.674 | 0.587 | 0.376 | 0.789 |
| \|difference\| | 0.062 | 0.063 | 0.000 | 0.000 |

$$\mathcal{L}_{\text{box}} = 0.062 + 0.063 + 0.000 + 0.000 = 0.125$$

> Loss 0.125. It is zero only if all four numbers match; gradient descent moves the four outputs toward the target.

▶ Notebook, part 3

# Two animals, one box: which one should it draw?




The annotation lists both animals. The head has room for one box.

Trained to match "the first animal in the list", and the list order is arbitrary, the safest single answer is in between: the average box.

$$\mathrm{IoU}(\mathrm{avg}, \mathrm{dog}) = 0.225$$
$$\mathrm{IoU}(\mathrm{avg}, \mathrm{cat}) = 0.114$$

It matches neither animal.

> The average box has IoU 0.23 with the dog and 0.11 with the cat. A box between two objects describes neither.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

# Give the head K slots




Problem 1: a photo with more objects than slots. The crowd photo has 11 people; 5 slots cannot hold them. So make K larger than any photo needs.

> Problem 1: what if a photo has more objects than K? Then choose K large. DETR uses K = 100.

▶ Notebook, part 3

# Problem 2: which dog belongs in slot 1?




$$\text{slot 1 vs entry 1:}\ \ \mathcal{L} = 1.212$$

$$\text{slot 1 vs entry 2:}\ \ \mathcal{L} = 0.012$$

The same correct answer costs 100 times more because of the list's order. The loss must not depend on the order.

> Nothing about the prediction changed, only the list order. A loss that pairs outputs with targets by position punishes correct answers.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 3

# Two ways to decide which output answers for which object

## A · by place

- Tie each output to a place in the image
- The outputs at the dog's place answer for the dog
- No slot ever chooses

Parts 4–6: YOLO

## B · by matching

- Keep free slots
- Before the loss, pair slots with objects in the cheapest way
- Each object gets exactly one slot

Part 7: DETR

> Fix B: keep free slots, and before each loss, match slots to objects so that the total cost is lowest. Then order cannot matter either.

▶ Notebook, part 4

# Part 4 · A candidate at every place



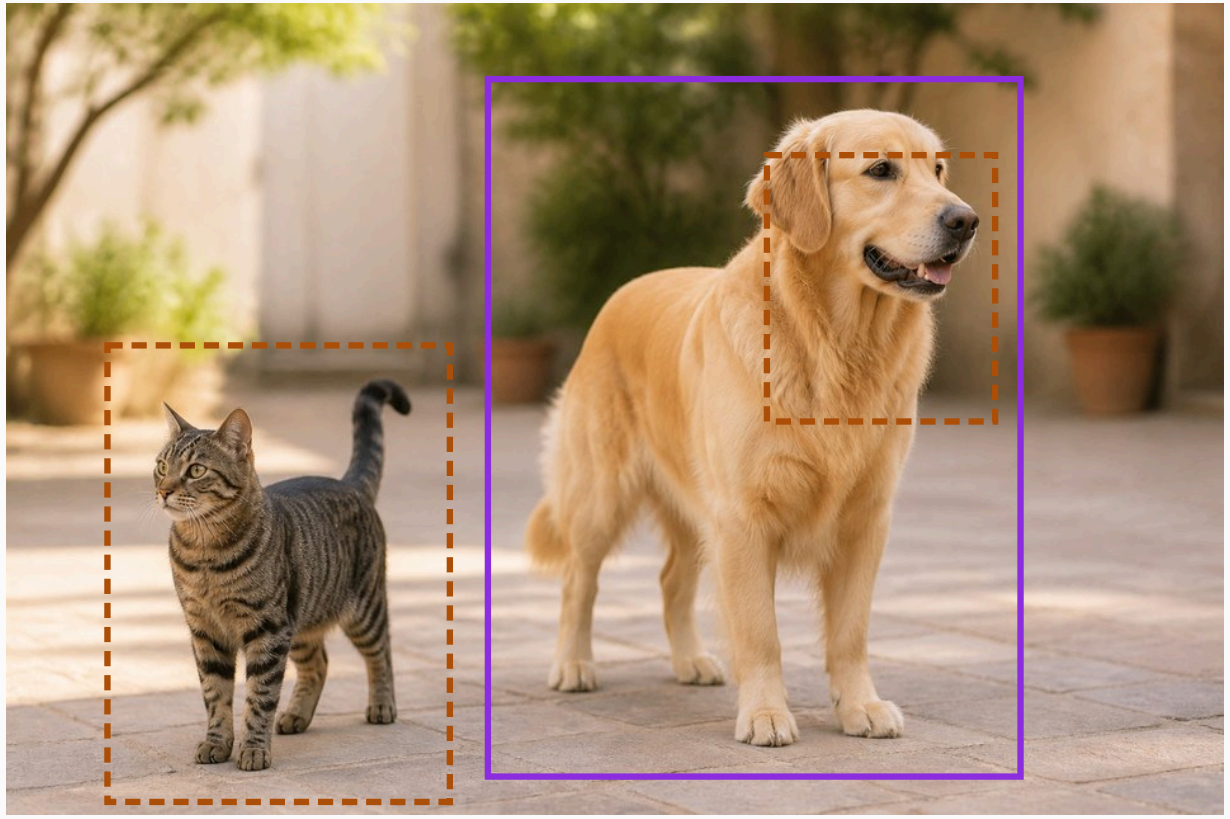
Tie each output to a place: a grid of cells over the photo, each writing one record.

1. Get a grid of feature vectors from one pass of a CNN; every cell writes a record of class scores and a box.
2. Decide which cells should answer for each object, and write their targets.
3. Compute the loss for one cell by hand, then read boxes back out at test time.

> A grid of cells, each writing a record: targets, loss, decoding.

▶ Notebook, part 4

# The brute-force idea: classify every window



Crop a window, run a classifier on it: is an object centred here? Which class?

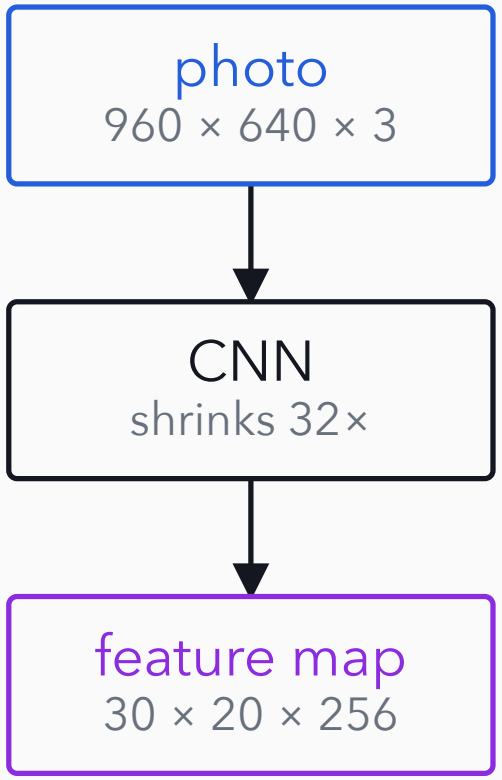
Every 32 px across and down: 30 × 20 = 600 positions, each at several sizes and shapes. Thousands of crops, each through a whole CNN.

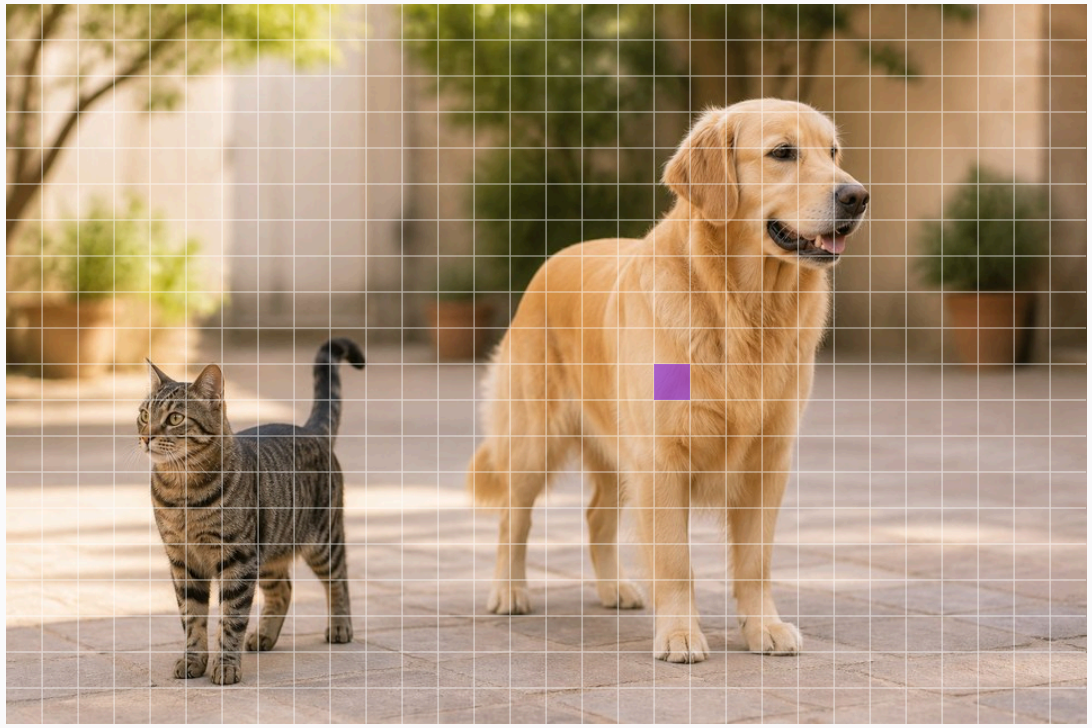
**A CNN already does this in one pass.** Its last feature map has one vector per position, and each vector describes the region around it.

> Convolution slides the same weights over every position anyway. One pass gives a vector for every position at once.

▶ Notebook, part 4

# One pass of a CNN: a 30 × 20 grid of feature vectors



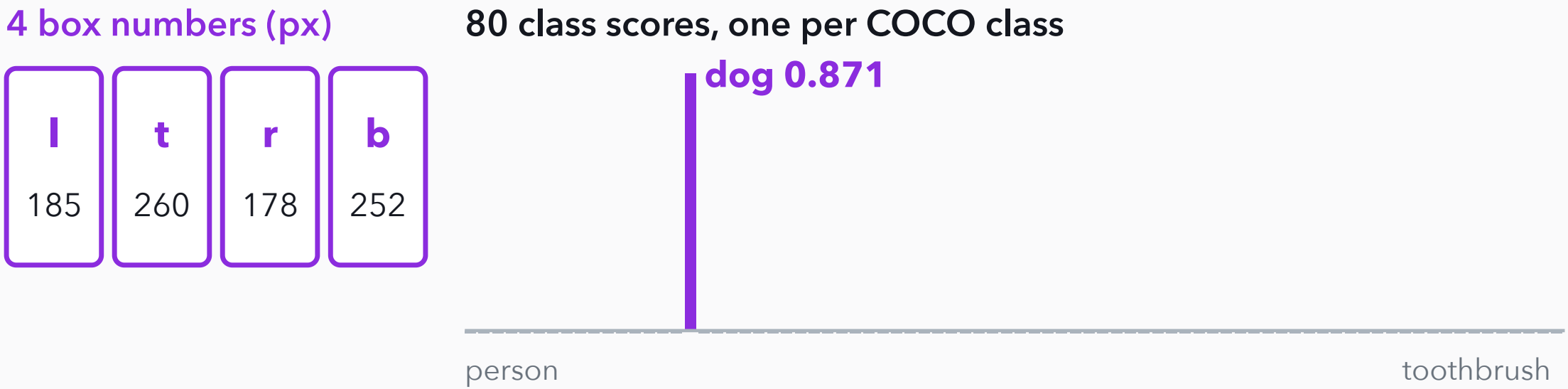



One cell = a 32 × 32 block of pixels: its address. Its vector is computed from a much larger region around it.

> The cell is an address, not a crop: its 256 numbers were computed from a region far larger than its block, wide enough to see the whole dog.

▶ Notebook, part 4

# Every cell writes a record: 4 box numbers, 80 class scores



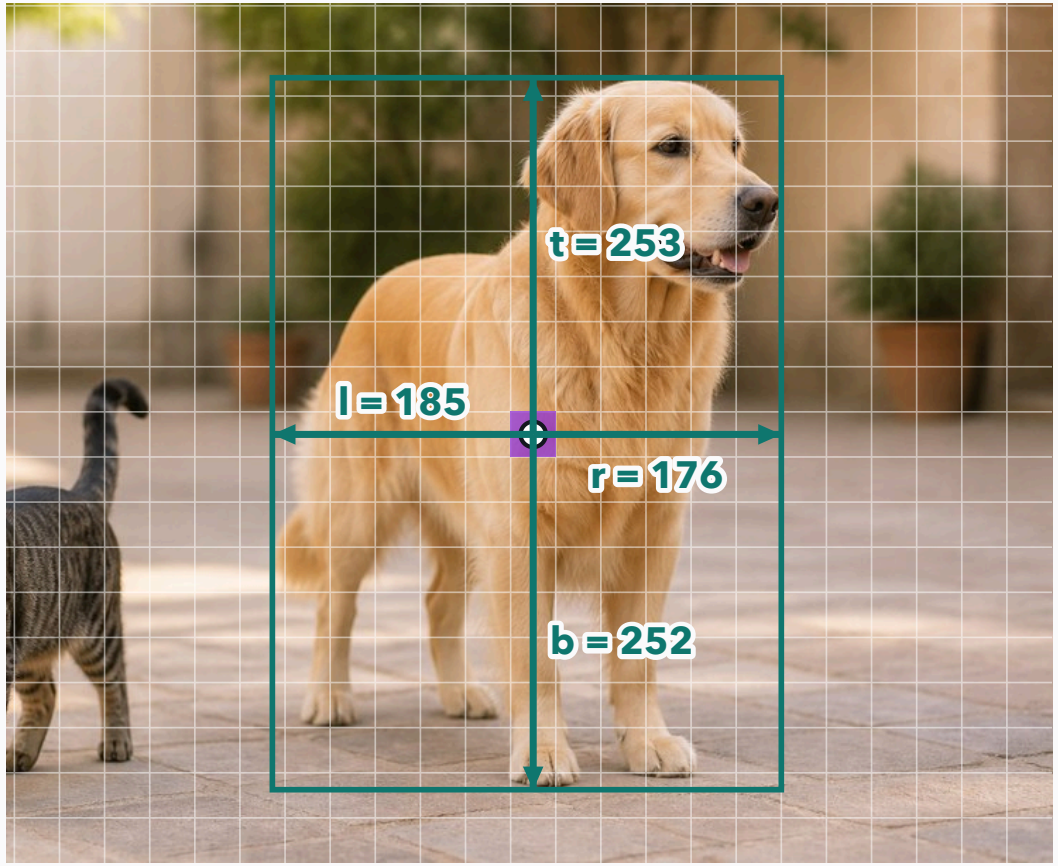

$$\underbrace{30 \times 20}_{\text{cells}} \times \underbrace{(4 + 80)}_{\text{record}} = 600 \times 84 \text{ numbers, for every photo of this size}$$

> The same small layer, with the same weights, writes this record at every cell: a 1 × 1 convolution. The output is a fixed 600 × 84 tensor.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 4

# How a cell describes a box: its distances to the four sides




$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

$$r = 768 - 592 = 176$$

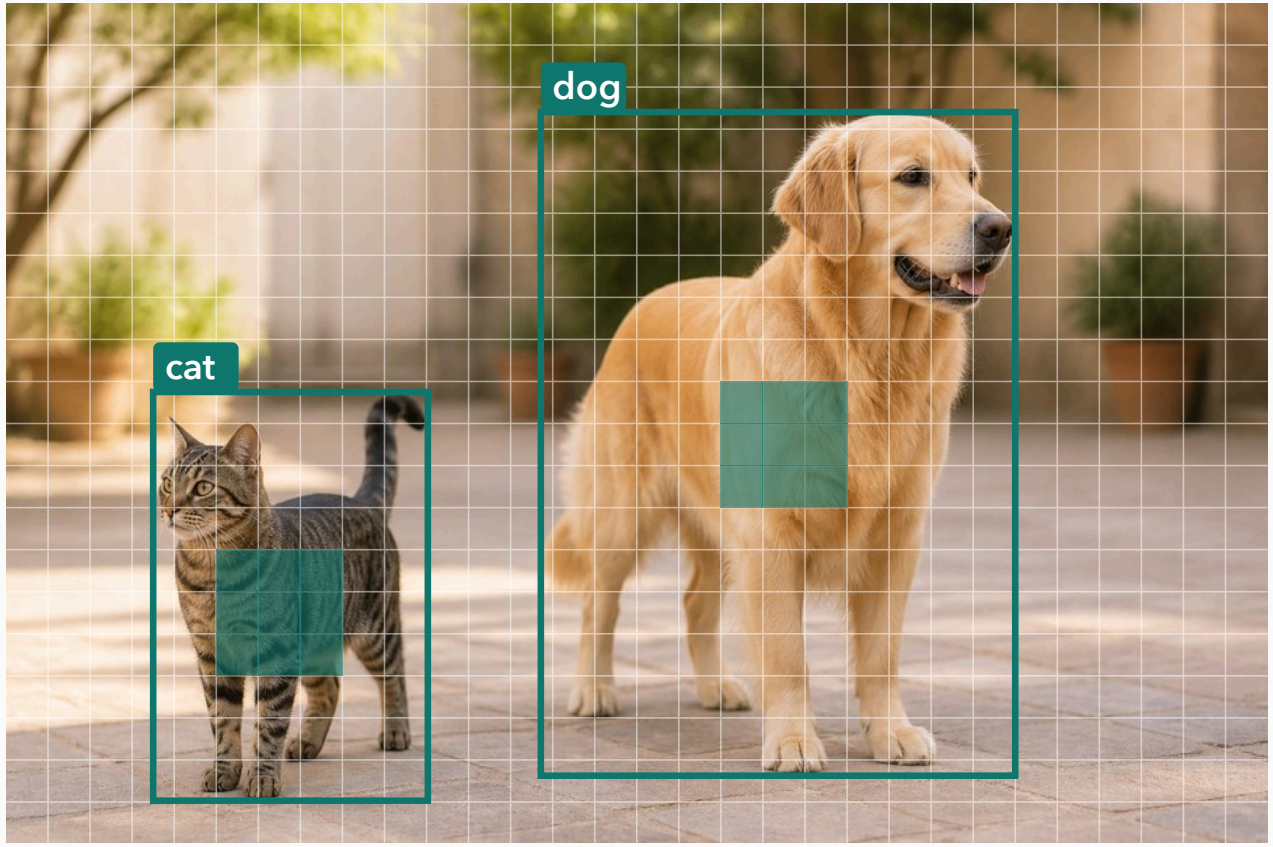
$$b = 588 - 336 = 252$$

> Bottom: 588 − 336 = 252. Like standing in a room and measuring the distance to each wall: four numbers fix the box.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

# Which cells should answer for the dog?




165 cells have their centre inside the dog's box. Should all of them predict the dog?

A common rule (FCOS's "centre sampling"): only cells within 1.5 cells of the box's centre. For the dog: rows 9–11, columns 17–19.

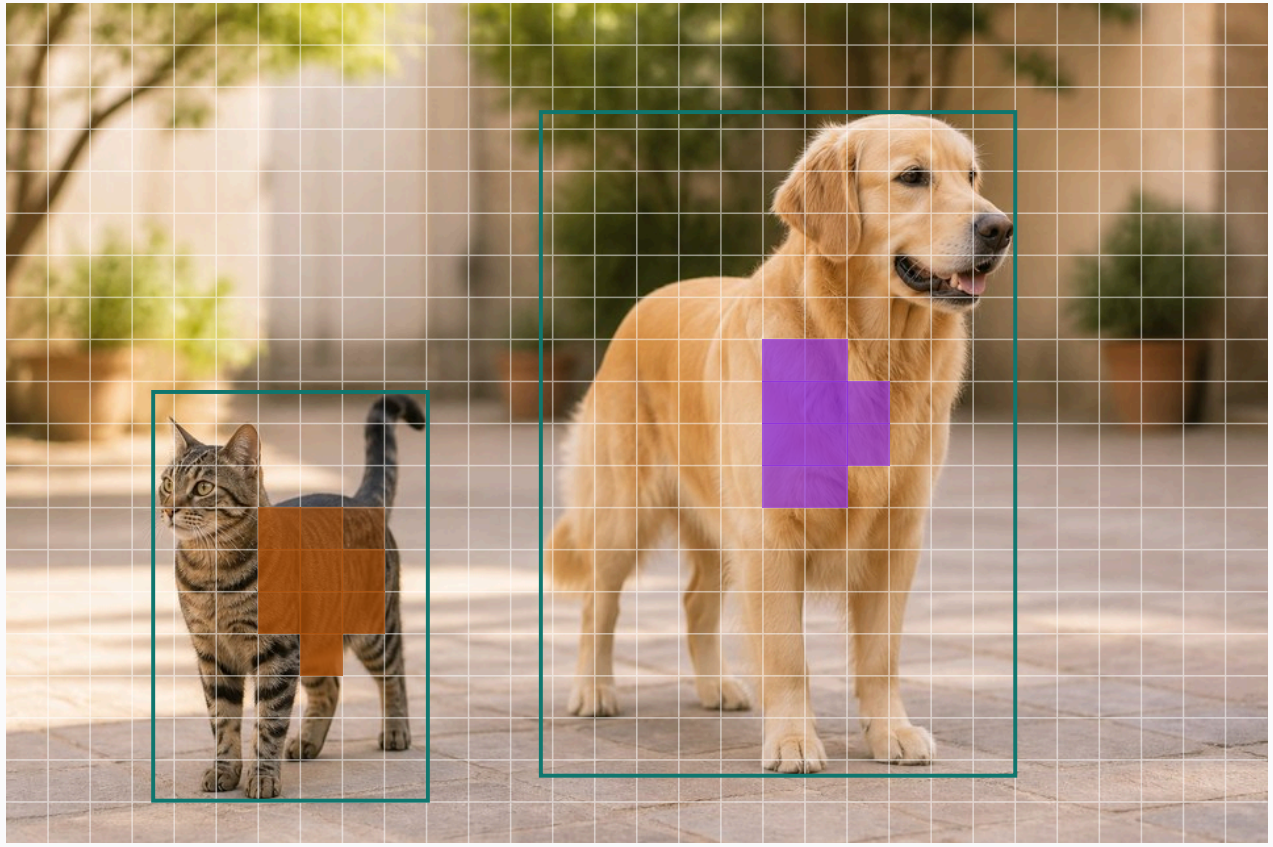
Same for the cat: rows 13–15, columns 5–7. These 18 cells are **positives**; the other 582 are **background**.

> Do the same for the cat. 18 cells get an object target; 582 get the target "no object of any class".

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

# The trained YOLOv8n: about ten cells fire per object



Shade = the cell's dog score. 10 cells score above 0.25, all near the dog's centre; 6 of them are in our 3 × 3 block.

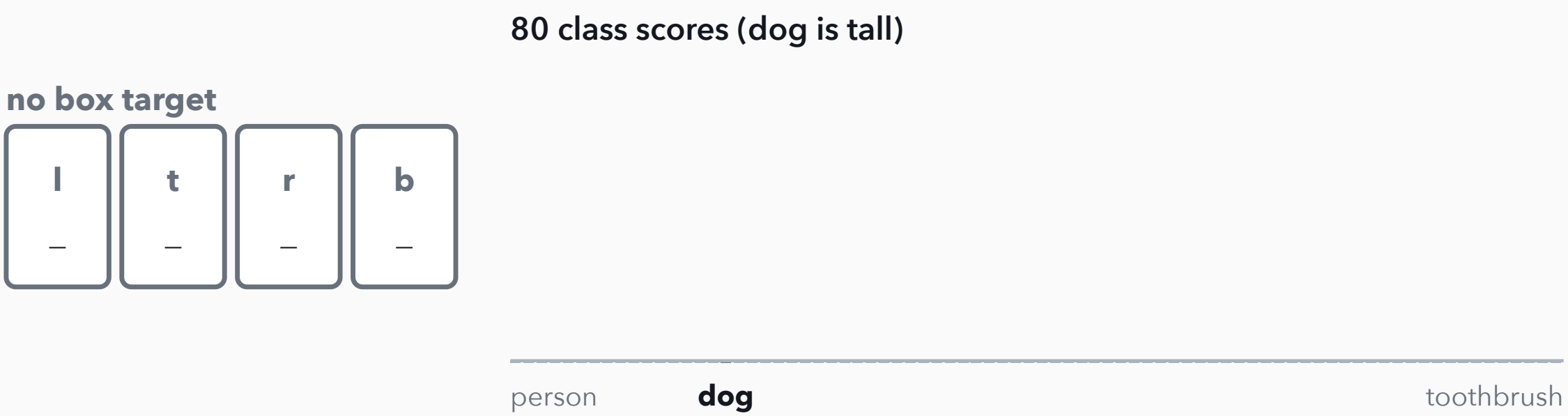
Cat scores: again 10 cells. YOLOv8 trains 10 positive cells per object, and the network learned to fire in about that many.

> The cat's scores make a second cluster. About ten cells per object: exactly what YOLOv8's training asked for, and the source of the duplicates we must remove later.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 4

# The target written at cell (10, 18)




A background cell, (3, 3) on the wall: all 80 class targets 0, and no box target at all.

> A background cell, such as (3, 3) on the wall: all 80 class targets are 0, and it has no box target, because there is no box to learn.

▶ Notebook, part 4

# What the network wrote at cell (10, 18)

**target**

| l | t | r | b |
|---|---|---|---|
| 185 | 253 | 176 | 252 |

**prediction**

| l | t | r | b |
|---|---|---|---|
| 184.8 | 260.4 | 177.6 | 252.1 |




Distances: close to the target, the top edge furthest off. The loss turns these differences into one number to reduce.

> Its four distances: (184.8, 260.4, 177.6, 252.1) against the target (185, 253, 176, 252). Close; the loss measures how close.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 4

# Class loss: one yes/no question per class

$$\mathrm{BCE}(p, y) = -\big[\, y \log p + (1 - y) \log(1 - p) \,\big]$$

$$\text{dog: } y = 1,\ p = 0.871: \quad -\log 0.871 = 0.139$$

$$\text{the other 79: } y = 0: \quad \sum_c -\log(1 - p_c) = 0.0004$$

$$\mathcal{L}_{\text{class}}(10, 18) = 0.139 + 0.0004 = 0.139$$

The cell's class loss: 0.139, almost all from the dog question.

▶ Notebook, part 4

# Box loss: how far off are the four distances?

$$\mathcal{L}_{\text{box}} = \sum_{k \in \{l,t,r,b\}} \left| \hat{d}_k - d_k \right|$$

$$= |184.8 - 185| + |260.4 - 253| + |177.6 - 176| + |252.1 - 252|$$

$$= 0.2 + 7.4 + 1.6 + 0.1 = 9.3 \text{ px}$$

$$1 - \text{IoU}(\text{predicted box}, \text{target box}) = 1 - 0.980 = 0.020$$

An IoU-based box loss is scale-free: a 9 px error matters more on a small box. YOLOv8 uses one (CIoU), plus a loss on how it spreads each distance.

> Another choice: one minus the IoU of the predicted and target boxes, here 0.020. It does not depend on the box's size, so small objects count as much as large ones.

▶ Notebook, part 4

# A background cell: only a class loss, and it is small

$$\text{cell } (3,3): \quad \sum_{c=1}^{80} -\log(1-p_c) = 0.0038 \qquad (\text{largest } p_c = 0.001,\ \text{potted plant})$$

But there are 582 background cells on this grid and only 18 positives. Tiny terms, many of them: detectors weight them down. YOLOv1 multiplied them by 0.5; RetinaNet's focal loss shrinks the easy ones (optional slide).

> The imbalance matters: hundreds of easy background terms against a handful of positives. Without care they dominate the sum and the network learns to say "nothing" everywhere.

▶ Notebook, part 4

# The loss for the whole photo

$$\mathcal{L} = \frac{1}{N_{\text{pos}}} \Bigg[ \underbrace{\sum_{\text{all cells}} \sum_{c=1}^{80} \text{BCE}(p_c, y_c)}_{\text{class: every cell}} + \lambda \underbrace{\sum_{\text{positive cells}} \mathcal{L}_{\text{box}}}_{\text{box: positives only}} \Bigg]$$

Here: 600 cells contribute class terms; the 18 positives also contribute box terms; divide by $N_{pos}$ = 18.

One number for the photo. Backpropagation sends its gradient into the shared head and the CNN; every cell's weights are the same weights, so every cell learns from every object.

> Gradient descent on this one number trains the shared layers.

▶ Notebook, part 4

# The same loss in code

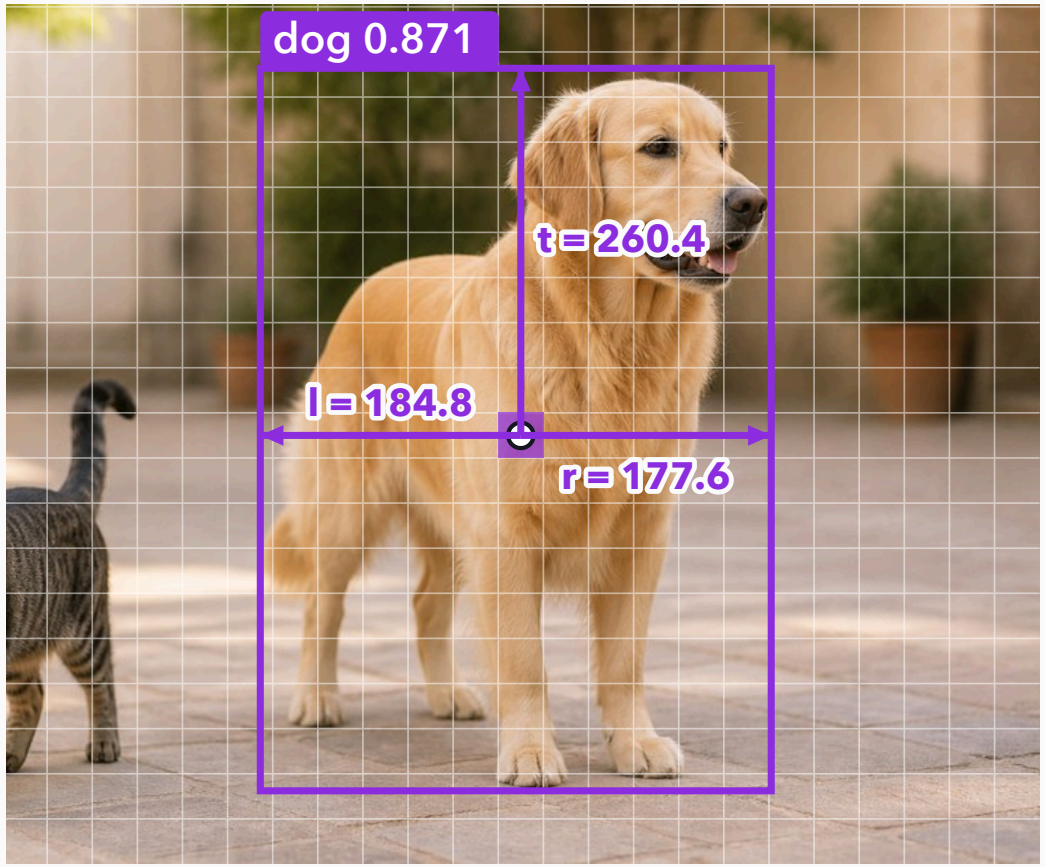
```
logits = head_cls(feats)                          # (600, 80): one row per cell
dist   = head_box(feats)                          # (600, 4): l, t, r, b in cells
y = torch.zeros(600, 80)
y[pos, labels] = 1.0                              # positives: 1 for their class
cls_loss = F.binary_cross_entropy_with_logits(logits, y, reduction='sum')
box_loss = (dist[pos] - dist_target[pos]).abs().sum()
loss = (cls_loss + box_loss) / len(pos)
```

pos: the indices of the 18 positive cells; labels: their class (16 for dog, 15 for cat); dist_target: their (l, t, r, b) targets.

> Building pos, labels and dist_target from the annotations is the “assignment” step: the 3 × 3 rule written as code. The notebook does it for this photo.

▶ Notebook, part 4

# At test time: read the box back out of the cell




No annotation now: only the record.

$$x_1 = 592 - 184.8 = 407.2$$
$$y_1 = 336 - 260.4 = 75.6$$
$$x_2 = 592 + 177.6 = 769.6$$
$$y_2 = 336 + 252.1 = 588.1$$

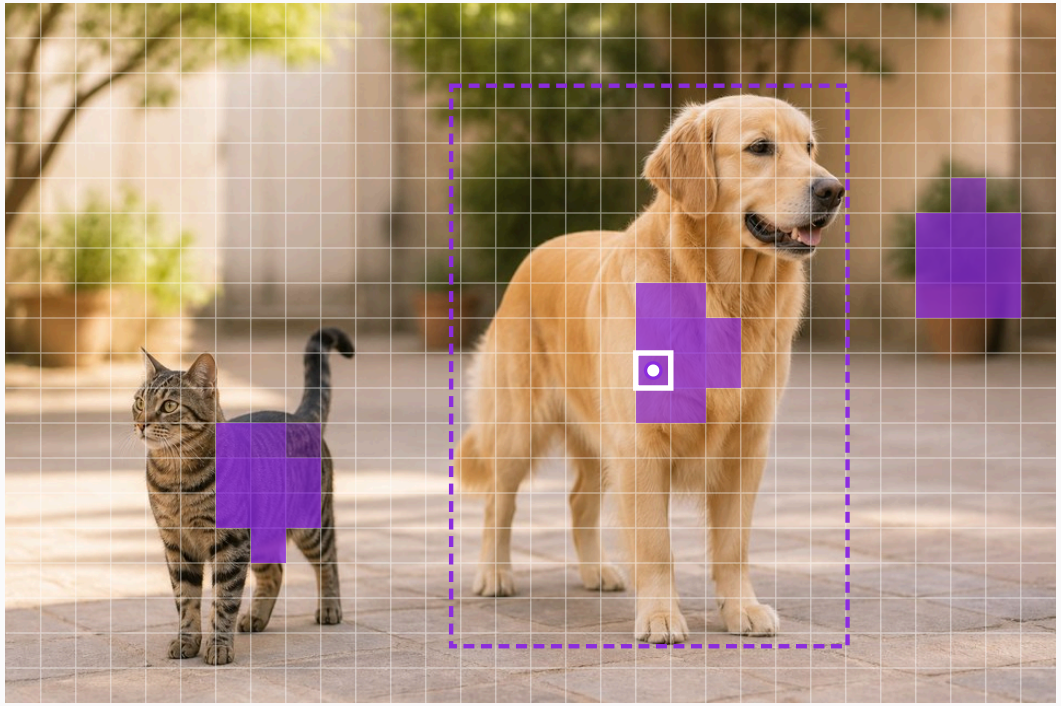
Score: the largest class score, 0.871, so the label is dog.

> Its score is its largest class score, 0.871, and its label that class. One cell, one scored box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 4

# Every cell writes a record: point at one



**Cell: row 10, column 18**

centre (592, 336) px

| | |
|---|---|
| dog | 0.871 |
| cat | 0.0001 |
| skateboard | 0.0001 |

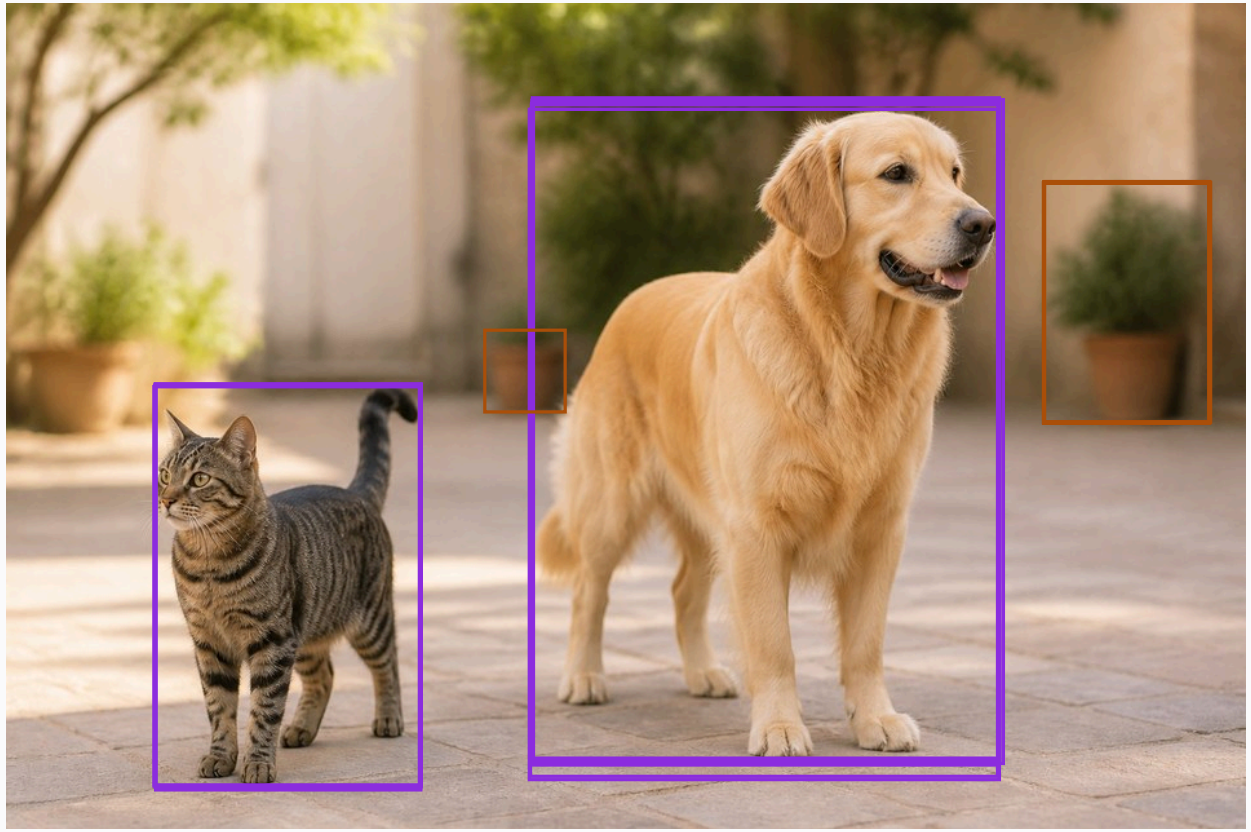
its box: (407, 76, 770, 588)

> Move over the grid. Cells near the dog say dog and draw nearly the same box; cells on the wall give every class a tiny score, and their boxes mean nothing.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 4

# Decode every cell, keep scores ≥ 0.25: 31 boxes



The network always writes 12,600 records for a photo this size: this grid's 600 and two finer grids' (Part 6).

Scores ≥ 0.25: 31 boxes. 10 on the dog, 10 on the cat, 11 on plants.

**We want one box per object.** Each animal has about ten nearly identical boxes, one per firing cell.

> The ten cells that fired for the dog each drew a dog box. To get one box per object, remove the duplicates.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# Part 5 · One box per object: non-maximum suppression



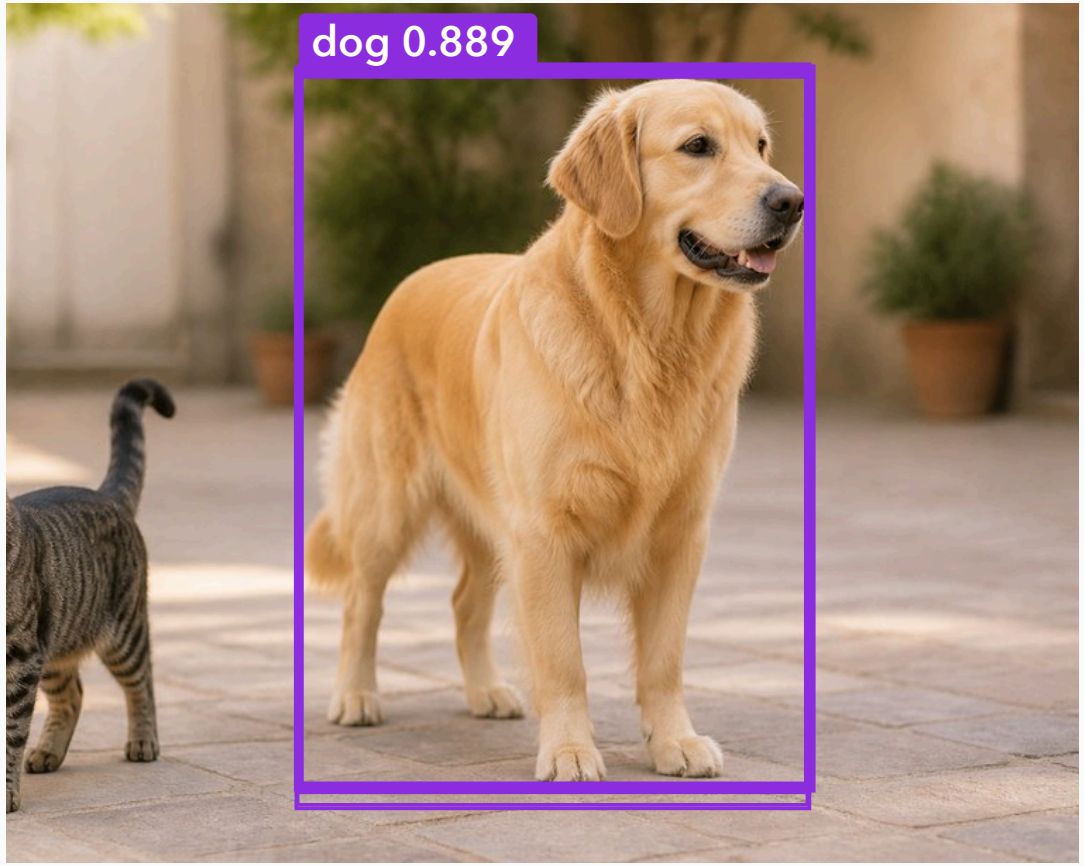
Ten boxes for one dog. Keep the best and delete the ones that overlap it heavily.

1. Run the greedy recipe on the dog's ten real boxes, one IoU at a time.
2. See what the IoU threshold trades off, and why classes are kept apart.
3. Try it live; write it in eight lines.

> NMS keeps the highest-scoring box and removes boxes that overlap it.

▶ Notebook, part 5

# Ten boxes for one dog




| cell (row, col) | score |
|---|---|
| (9, 19) | **0.889** |
| (9, 18) | 0.888 |
| (10, 18) | 0.871 |
| (10, 19) | 0.862 |
| (8, 18) | 0.822 |
| (10, 20) | 0.797 |
| (8, 19) | 0.793 |
| (11, 19) | 0.792 |
| (9, 20) | 0.792 |
| (11, 18) | 0.766 |

> They describe one dog. Keep the most confident box, from cell (9, 19), and delete the rest.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).
4. Repeat from step 2 with what is left, until nothing is left.

> Then repeat: the best of what is left is kept, and so on. “Non-maximum suppression”: suppress what is not the local maximum.

▶ Notebook, part 5

# NMS on the dog's ten boxes

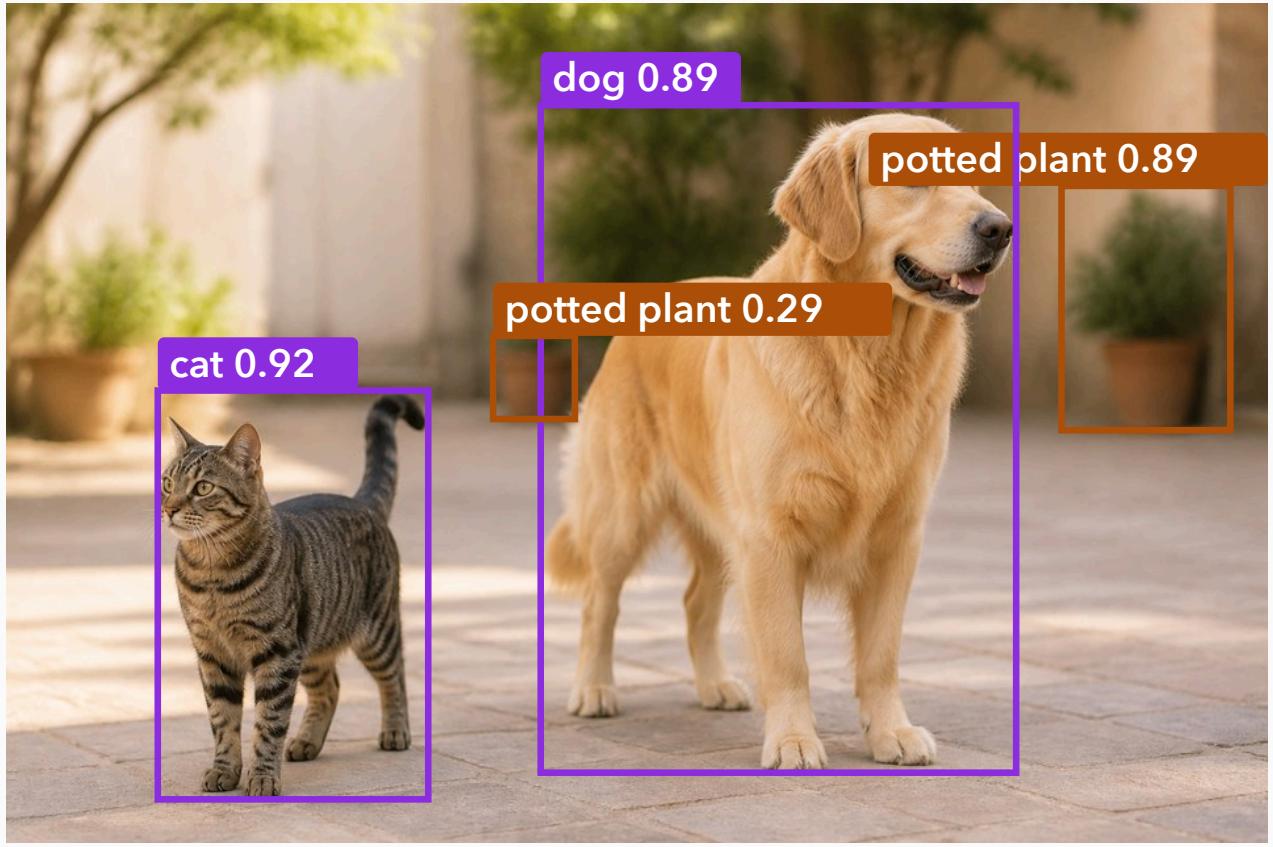
| rank | cell | score | IoU with the kept box | fate |
|---|---|---|---|---|
| 1 | (9, 19) | 0.889 | kept box | **keep** |
| 2 | (9, 18) | 0.888 | 0.993 | **delete (> 0.7)** |
| 3 | (10, 18) | 0.871 | 0.988 | **delete (> 0.7)** |
| 4 | (10, 19) | 0.862 | 0.985 | **delete (> 0.7)** |
| 5 | (8, 18) | 0.822 | 0.961 | **delete (> 0.7)** |
| 6 | (10, 20) | 0.797 | 0.981 | **delete (> 0.7)** |
| 7 | (8, 19) | 0.793 | 0.956 | **delete (> 0.7)** |
| 8 | (11, 19) | 0.792 | 0.980 | **delete (> 0.7)** |
| 9 | (9, 20) | 0.792 | 0.996 | **delete (> 0.7)** |
| 10 | (11, 18) | 0.766 | 0.977 | **delete (> 0.7)** |

> All above the 0.7 threshold: delete all nine. Nothing left for the dog; one dog box remains.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# All 31 boxes → 4 detections




Before: 31 boxes.

After: 4. cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290.

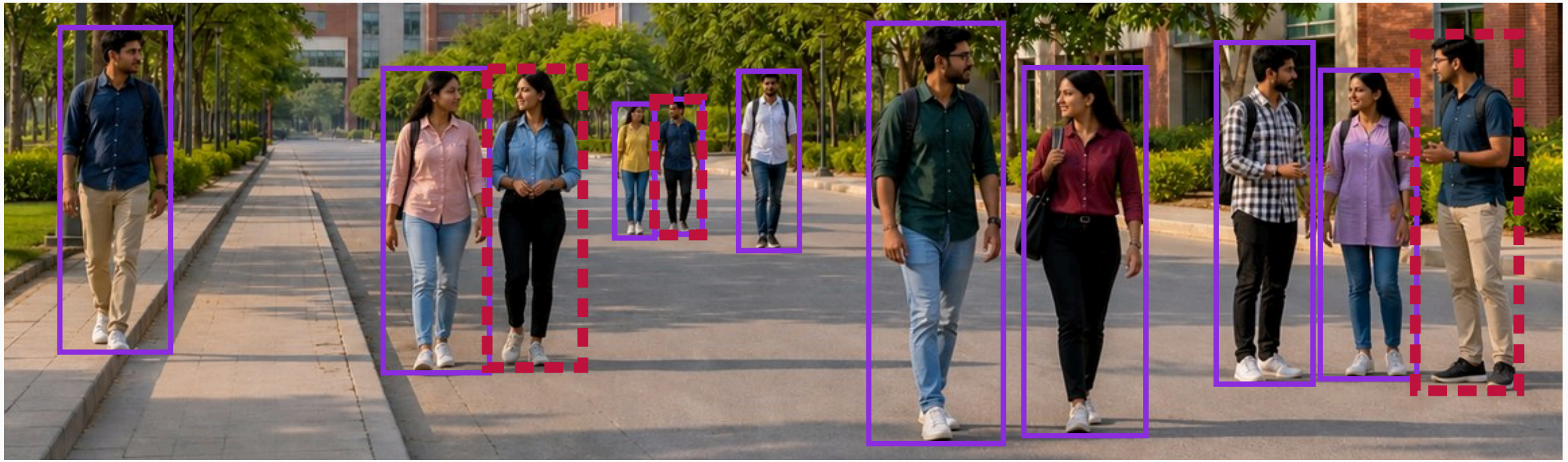
Our eight-line loop keeps exactly what Ultralytics' own post-processing keeps (checked in scripts/science.py).

> Four detections: cat, the big potted plant, dog, and a small plant on a finer grid. A fixed 12,600 records became a set of four.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# The IoU threshold is a trade-off



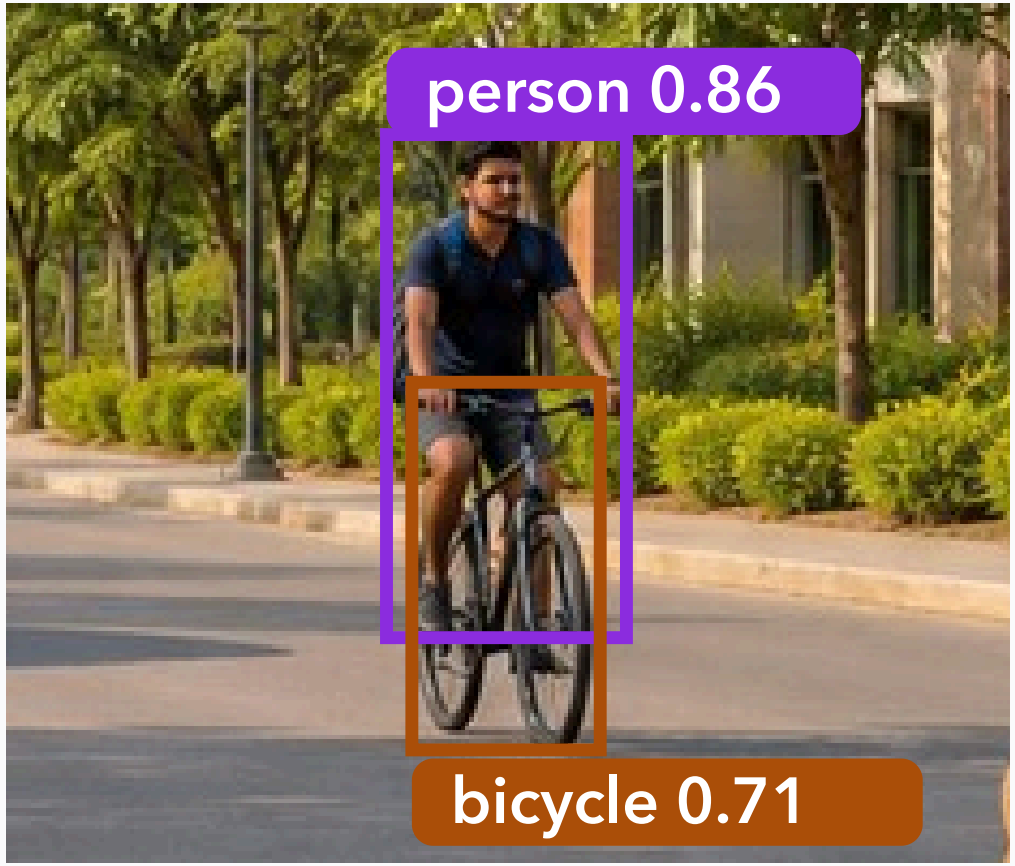
Too low, 0.0 (any overlap at all): 9. Two people whose boxes merely touch a neighbour's are deleted (red).

> Lower it to 0 and any touching box is deleted, including real neighbours. People in a queue or a crowd overlap far more than this; no single threshold suits every scene.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# Boxes of different classes do not suppress each other




$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

Ignore classes, threshold 0.3: the bicycle overlaps the higher-scoring person by more than 0.3, so it is deleted.

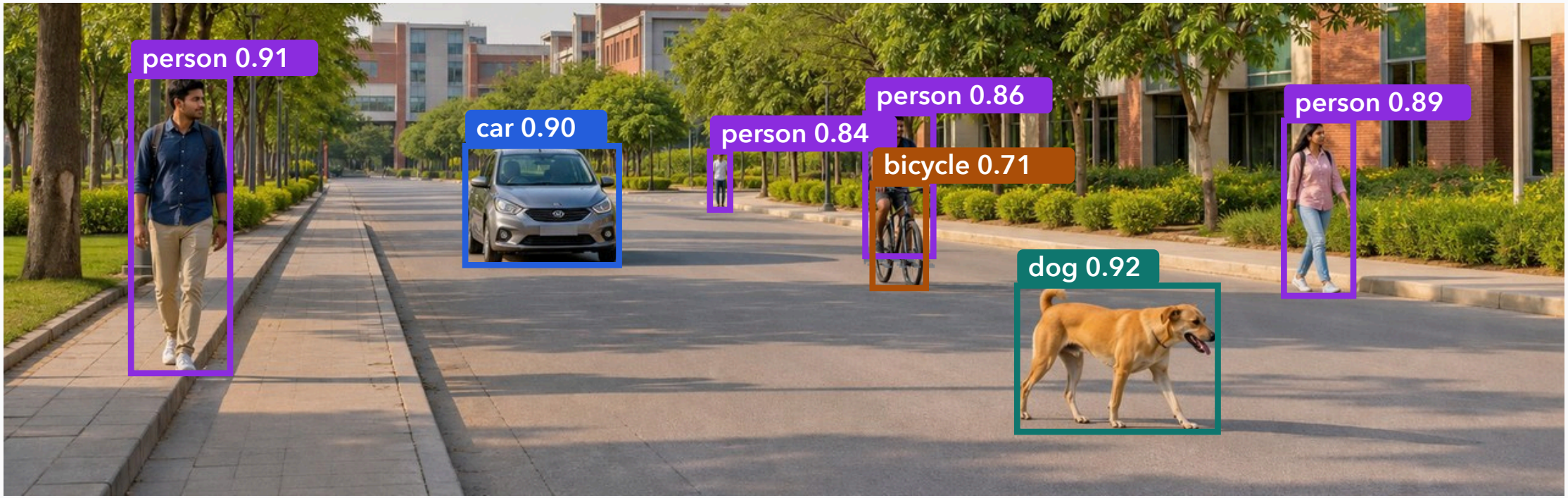
Class-aware NMS only compares boxes of the same class: both survive. This is the usual choice (torchvision's batched_nms).

> Run NMS separately for each class and both are kept. Steps 3 of the recipe said "of the same class" for this reason.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# Try it: thresholds on the street photo




> Real candidates from YOLOv8n. Move the score threshold, then the IoU threshold; switch to “all classes together” and watch the bicycle; press Step to run the loop one round at a time.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

# NMS in code

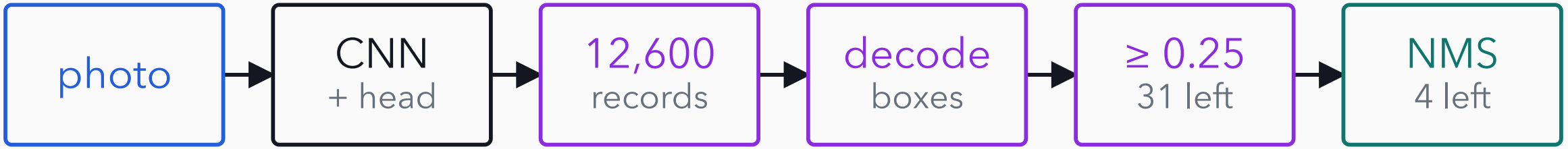
```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)          # step 1
    keep = []
    while order.numel() > 0:
        i = order[0]; keep.append(i)                 # step 2
        ious = box_iou(boxes[i, None], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]               # step 3, then repeat
    return torch.stack(keep)
```

```python
torchvision.ops.nms(boxes, scores, 0.7)              # same indices
torchvision.ops.batched_nms(boxes, scores, labels, 0.7)  # per class
```

> torchvision has it built in; batched_nms runs it separately for each class label.

▶ Notebook, part 5

# The dense answer, end to end




**Fixed size in, any size out.** The network's output never changes shape; the score filter and NMS decide how many objects come out: here 4, for the crowd 11, for an empty wall 0.

> This is answer A to the lecture's question: place-based outputs, many per object in training, one per object after NMS.

▶ Notebook, part 6

# Part 6 · Small and large objects: three grids



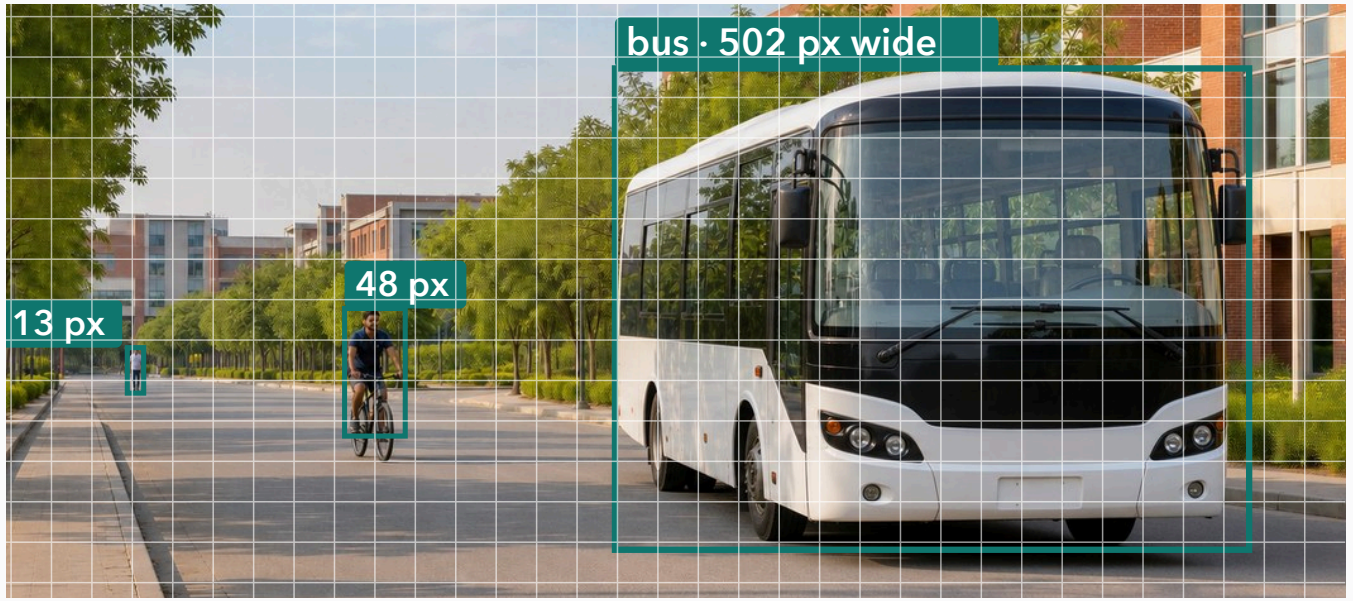
A 32-pixel cell is coarse for a person 13 pixels wide. YOLOv8 predicts from three grids at once.

1. See the range of sizes in one street photo.
2. Count the cells of the three grids: where 12,600 came from.
3. Check which grid found each real detection.

> Three grids, three cell sizes, one record format.

▶ Notebook, part 6

# One photo, very different sizes




The bus is 502 px wide; the far pedestrian 13 px.

On a 32-px grid the pedestrian is under half a cell. Two such people side by side would put both centres in one cell, which can describe only one of them.

> Draw the 32-px grid. The bus spans 16 cells; the pedestrian sits inside one. A coarse grid suits big objects; small ones need finer cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 6

# Three grids: cells of 8, 16 and 32 pixels

| grid (cell size) | courtyard, 960 × 640 | street, 1280 × 704 | built from |
|---|---|---|---|
| 8 px | 120 × 80 = 9,600 | 160 × 88 = 14,080 | early layers: sharp, local |
| 16 px | 60 × 40 = 2,400 | 80 × 44 = 3,520 | middle layers |
| 32 px | 30 × 20 = 600 | 40 × 22 = 880 | deep layers: wide view |
| **all** | **12,600** | **18,480** | |

Every cell of every grid writes the same 84-number record. The fine grid catches small objects; the coarse grid sees enough to box a bus.

> Each grid writes the same kind of record. Feature pyramids (FPN, 2017) mix deep and early features so even the fine grid knows what it is looking at.

▶ Notebook, part 6

# Which grid found each object? The real answer




| object | grid |
|---|---|
| bus | 32 px |
| person | 16 px |
| person | 8 px |
| bicycle | 16 px |

Street photo: dog and near person from 32 px; car, cyclist and bicycle from 16 px; the far person from 8 px.

> The street photo agrees: large from coarse, small from fine. Training assigns each object to cells whose grid suits its size.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

# Part 7 · DETR: a fixed set of slots, matched to objects



Back to Part 3's slots: make 100 of them, let each read the whole image, and match slots to objects before the loss.

1. Follow a photo through DETR to 100 predictions, and see where two queries looked.
2. Watch duplicates vanish inside the decoder.
3. Build the matching cost one cell at a time, solve the matching, and compute the loss.

> DETR: 100 slots that read the image with attention; matching instead of NMS.

▶ Notebook, part 7

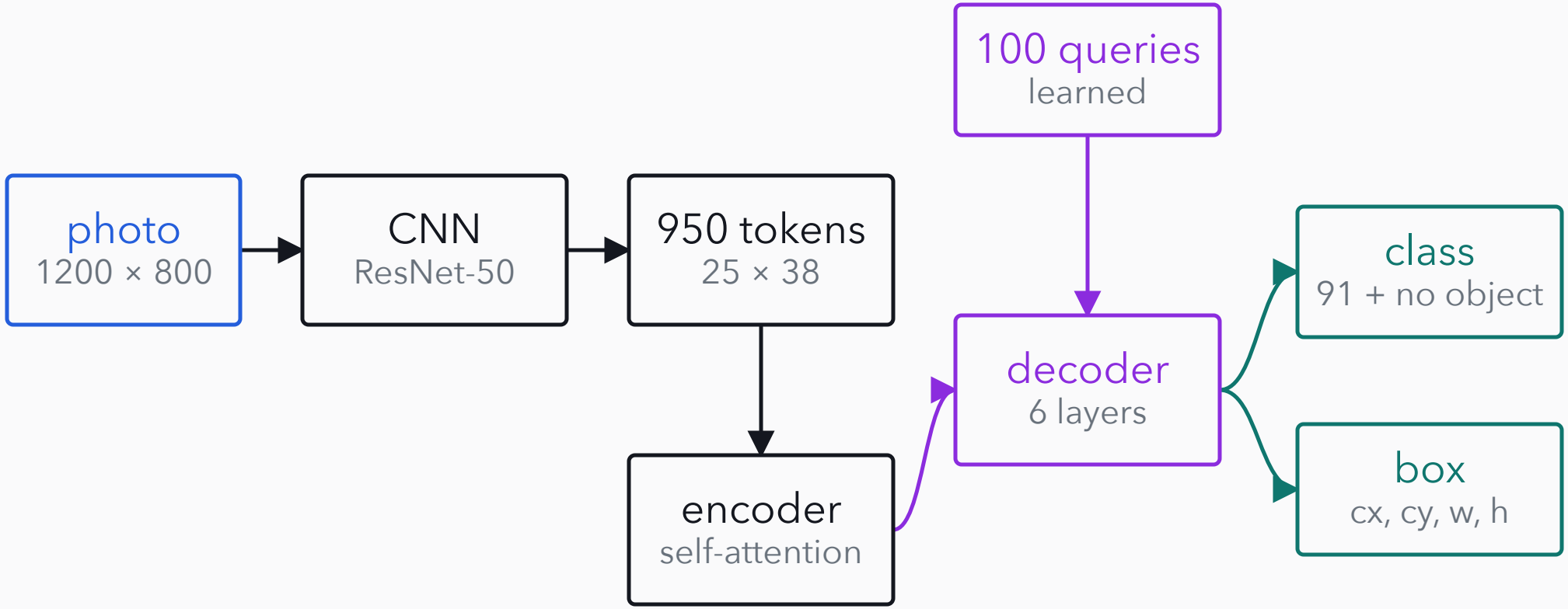
# Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.
3. **Match before the loss**: pair slots with objects so that the total cost is lowest.

> Problem 2 was order: before computing the loss, find the cheapest one-to-one pairing of slots and objects. Order in the annotation list stops mattering.

▶ Notebook, part 7

# DETR, from photo to 100 predictions




> Two small heads read each of the 100 outputs: a class among 91 COCO ids plus “no object”, and a box as normalised (c_x, c_y, w, h).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# A query is learned once and reused for every photo

**Stored:** 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

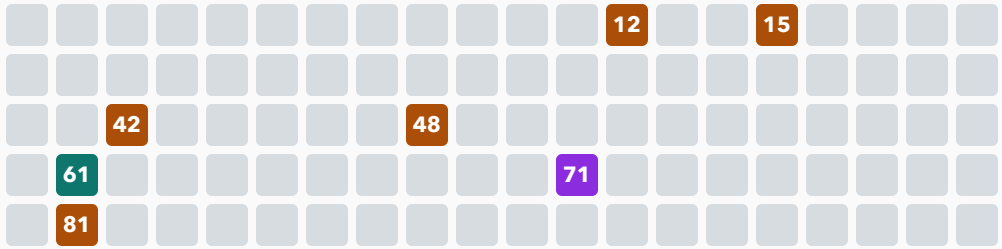
**Computed:** what each query writes depends on the photo, because it reads the photo through cross-attention.

As in the VLM lectures: stored query slots, computed per-image summaries.

> Their outputs are computed: the same query writes a dog box for one photo and “no object” for another, depending on what it read.

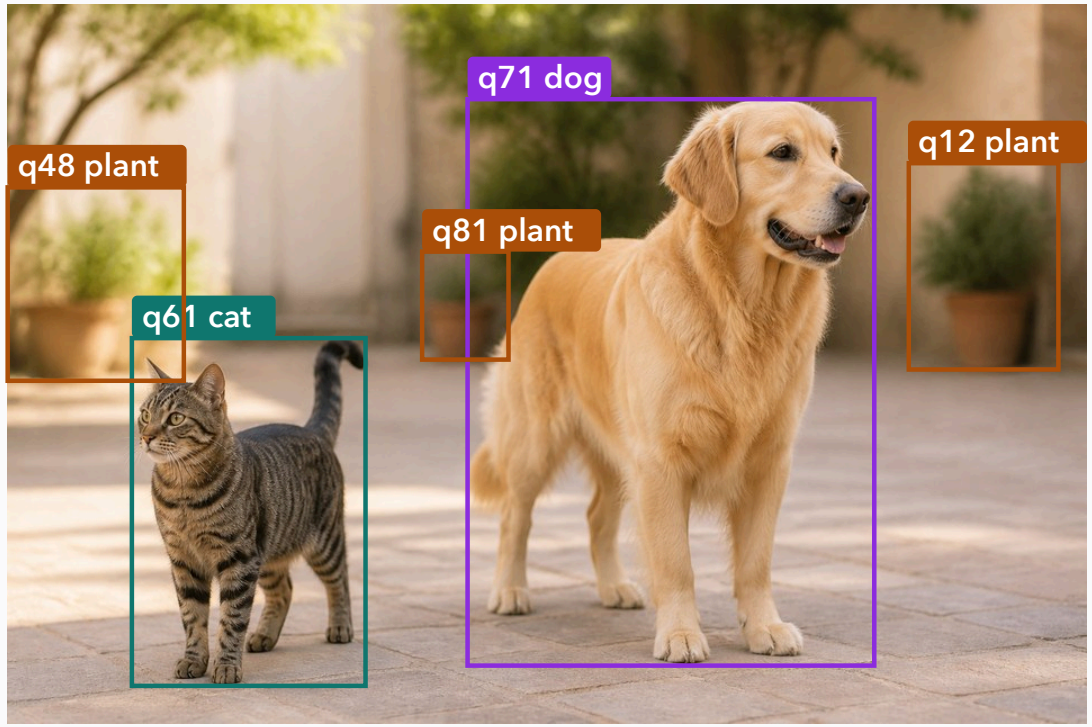
▶ Notebook, part 7

# DETR on the courtyard: 100 slots, most of them empty




Grey: “no object”. 88 of the 100 slots give it more than 0.9.

| query | class | p | p(no object) |
|---|---|---|---|
| 71 | dog | 0.999 | 0.001 |
| 61 | cat | 0.999 | 0.001 |
| 12 | potted plant | 0.998 | 0.002 |
| 81 | potted plant | 0.989 | 0.011 |
| 48 | potted plant | 0.973 | 0.027 |




> Their boxes: one per object. No duplicates to remove, and no NMS.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Where two queries looked



query 71 → dog



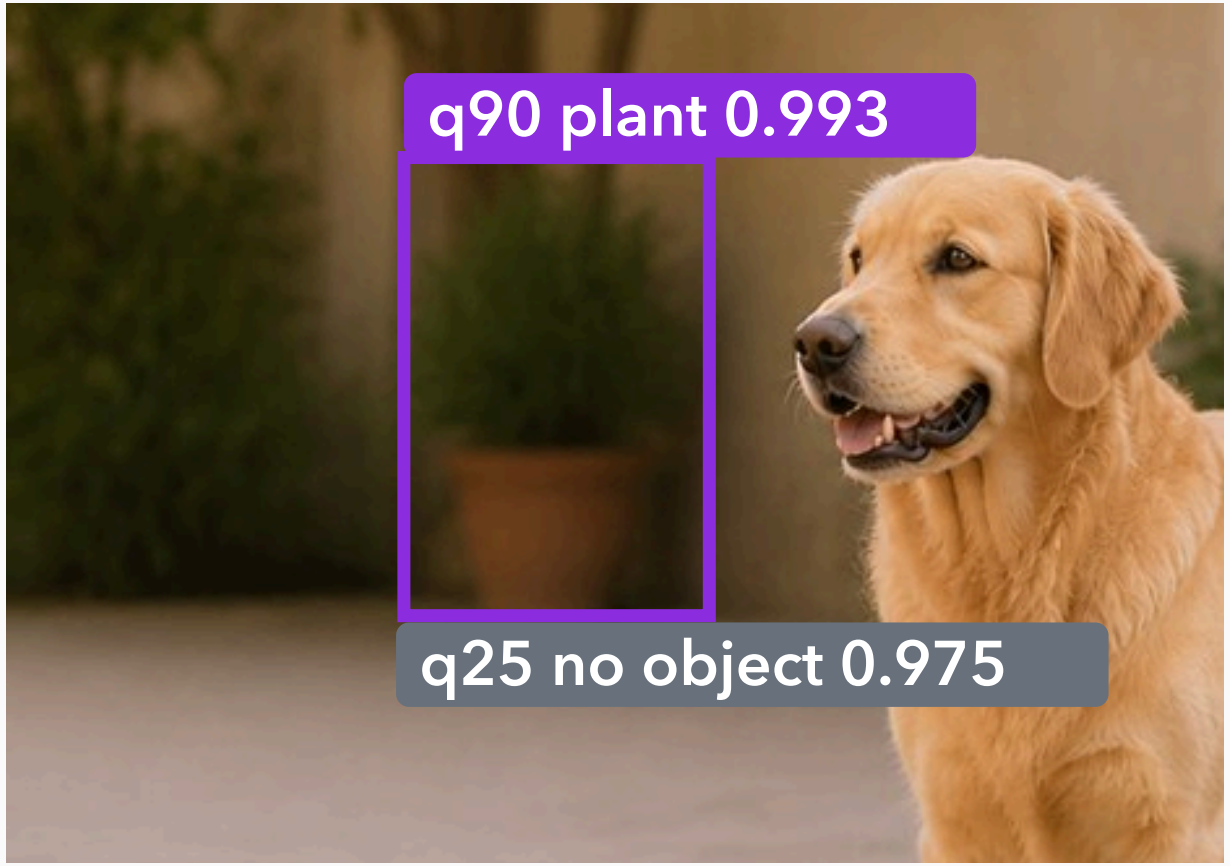
query 61 → cat

> Query 61, the cat: its attention sits on the cat. Each query attends mostly to the parts of the photo that fix its box: the extremities more than the middle.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Duplicates disappear inside the decoder




After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

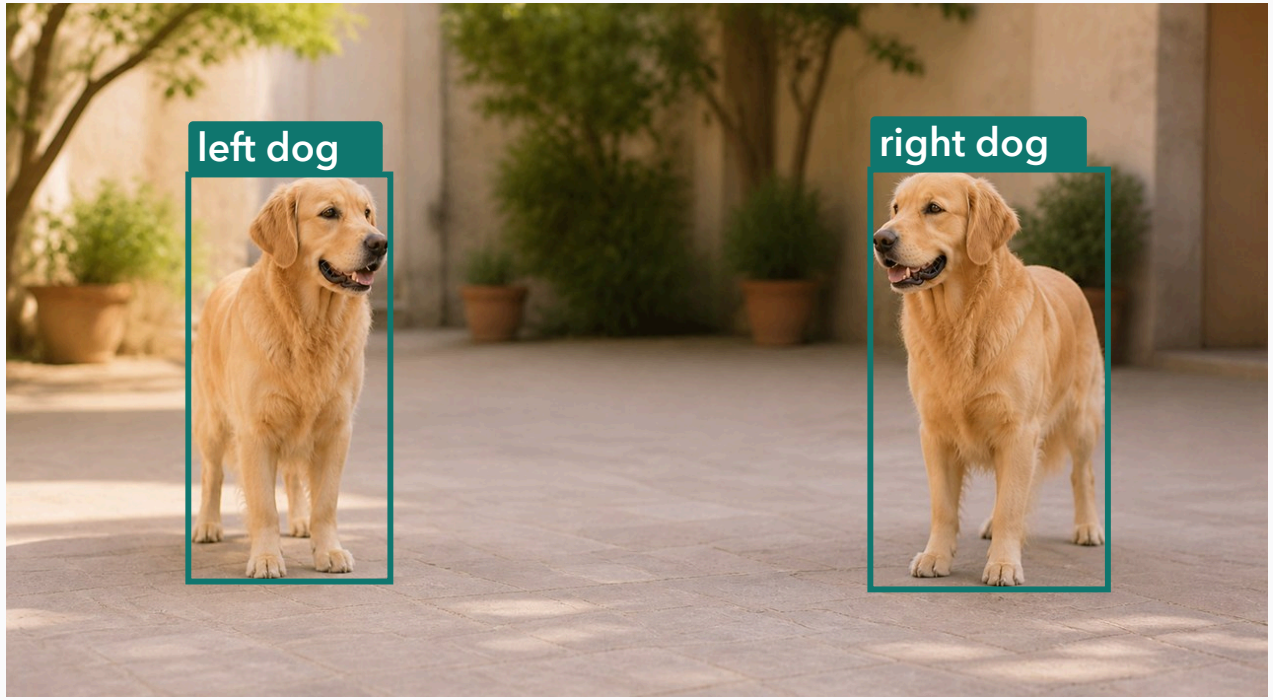
Queries attend to each other in every decoder layer, so one can see that another already has the pot. Training rewards exactly one of them per object (next slides).

> This works because training never rewards two queries for one object. That comes from the matching.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Training: which prediction should learn which dog?




Two targets. 100 predictions. Part 3: comparing slot k with list entry k punished a correct answer.

Instead: give every (prediction, target) pair a **cost**, then choose the one-to-one pairing with the lowest total.

> Score every pairing with a cost: low when the prediction has the right class and a box close to the target's. Then pick, among all one-to-one pairings, the cheapest.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

# One more measure: generalised IoU




Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

DETR's right-dog query against the left dog: IoU 0, GIoU −0.529.

> Real example: query 98 predicts the right dog; against the left dog its IoU is 0 but its GIoU is −0.529.

▶ Notebook, part 7

# The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

$$C = -0.999 + 0.020 - 1.968 = -2.95$$

> Total −2.95. Low cost = good pairing; negative is fine, only comparisons matter.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Fill the table: the cost of every pairing

| | q61 (dog) | q98 (dog) | q90 (plant) | q2 (no object) |
|---|---|---|---|---|
| left dog | **−2.95** | 3.09 | 6.07 | −1.71 |
| right dog | 3.08 | **−2.90** | 4.47 | 4.26 |

Cheapest pairing: left dog ↔ q61, right dog ↔ q98, total −5.85. Every other query is matched to nothing and learns “no object”.

q2’s box also sits on the left dog (cost −1.71), but it says “no object”: the class term makes q61 cheaper.

> Choose one query per dog with the lowest total: q61 and q98. Here each dog’s cheapest query is different, so the choice is easy. It is not always.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Why not let each object grab its cheapest slot?

| | q1 | q2 |
|---|---|---|
| left dog | 1 | **2** |
| right dog | **2** | 9 |

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

Trying every pairing is hopeless with 100 queries. The **Hungarian algorithm** (Kuhn, 1955) finds the cheapest one-to-one pairing in polynomial time: `scipy.optimize.linear_sum_assignment`.

> The best pairing must be found globally. The Hungarian algorithm does it efficiently; scipy has it.

▶ Notebook, part 7

# After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \;\; 0.1 \times (-\log 0.997) = 0.0003$$

$$\text{q90 (a plant)} \to \text{no object:} \;\; 0.1 \times (-\log 0.003) = 0.581$$

Our target list holds only the two dogs, so the loss would teach q90 to call the pot nothing. What is annotated decides what is learned; COCO does label potted plants.

> q90 named a real plant, but plants are not in this target list: it pays 0.58 for being right. Annotation policy matters.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 7

# Why DETR needs no NMS

## DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

## YOLO · one-to-many

- About ten cells per object are trained as positives
- All ten learn to fire
- So NMS must delete nine of them

> The dense detector trained ten cells per object on purpose (more positives, more signal) and pays for it with NMS.

▶ Notebook, part 7

# DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                    # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())          # one query per object
```

```
model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
out = model(**processor(images=img, return_tensors="pt"))
dets = processor.post_process_object_detection(out, threshold=0.9,
                                               target_sizes=[img.size[::-1]])
```

> At test time there is nothing to match: run the model, keep the queries whose best class beats a threshold. No NMS.

▶ Notebook, part 7

# Two answers to one question

| | dense (YOLOv8) | set (DETR) |
|---|---|---|
| outputs | one record per cell: 12,600 | 100 learned slots |
| who answers for an object | cells near it, by place | the matched query |
| training targets | one-to-many: ~10 cells | one-to-one: Hungarian matching |
| at test time | score filter + NMS | threshold only |
| on our courtyard | 4 detections | 5 objects at $p \geq 0.9$ |

Both turn a fixed-size output into a set of any size: by scores and NMS, or by “no object” and matching.

> Both solve the lecture’s question with a fixed tensor plus a way to say “nothing here”.

▶ Notebook, part 8

# Part 8 · Measuring a detector

1 · A list, not a label | 2 · Boxes and IoU | 3 · One box, then many | 4 · A dense detector | 5 · NMS | 6 · Three grids | 7 · DETR

8 · Measuring

Compare detections with annotations: decide which are right, then summarise.

1. Decide true positive, false positive and miss, on the street photo's real detections.
2. Precision and recall at one threshold, then at all of them: walk down a ranked list.
3. Average precision, mAP, and both detectors on our eight photos.

> Right or wrong for each detection, then precision, recall and AP.

▶ Notebook, part 8

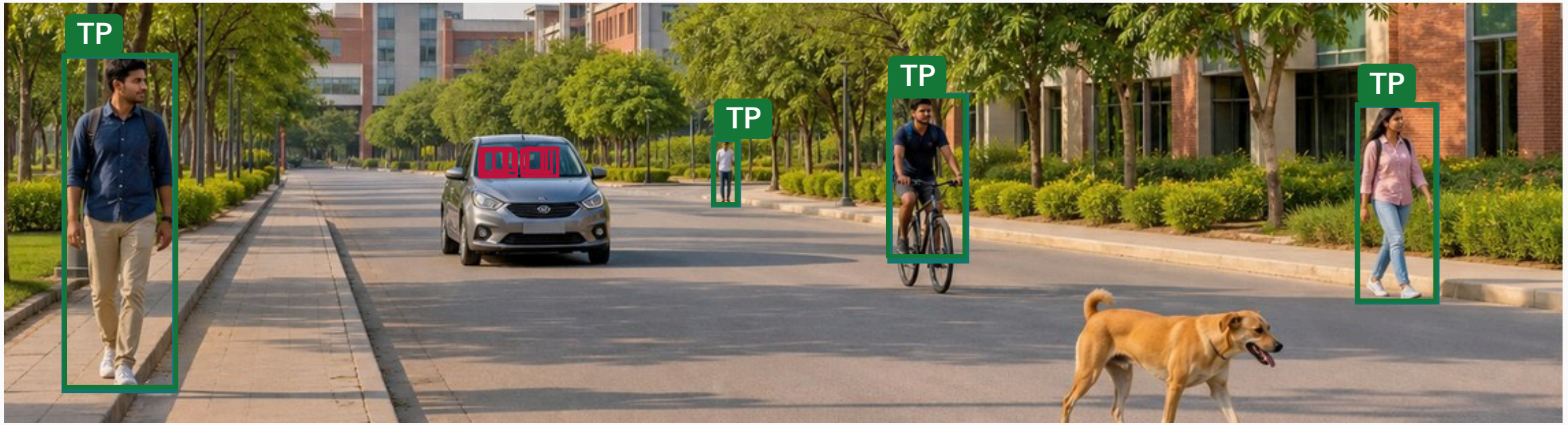
# When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.
4. **False negative (FN)**: an annotated object that no detection claimed: a miss.

> Every annotated object left unclaimed is a false negative.

▶ Notebook, part 8

# Real check: people in the street photo




The other 8 (scores 0.042 down to 0.011) sit on the car's windscreen, IoU 0 with every person: **8 FP**. No person is missed: 0 FN.

> The other 8, scores 0.04 and below, sit on the car's windscreen, where nobody is: false positives. No person is missed.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

# Precision and recall

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Precision: of what we reported, how much was right. Recall: of what was there, how much we found.

| keep scores ≥ | TP | FP | FN | precision | recall |
|---|---|---|---|---|---|
| 0.25 | 4 | 0 | 0 | 4/4 = 1 | 4/4 = 1 |
| 0.01 | 4 | 8 | 0 | 4/12 = 0.33 | 4/4 = 1 |

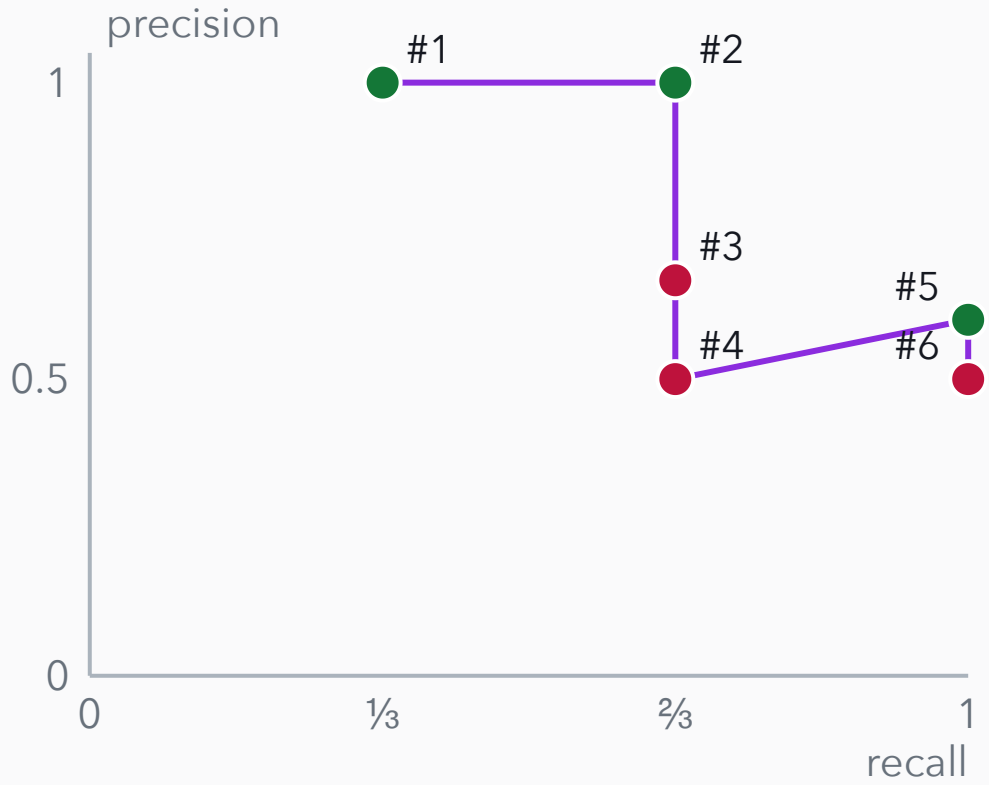
> Keep everything down to 0.01 and the eight windscreen boxes join: recall stays 1, precision drops to 0.33. The threshold trades one for the other.

▶ Notebook, part 8

# Walk down the ranked list, one detection at a time

| rank | score | outcome | why | precision | recall |
|---|---|---|---|---|---|
| #1 | 0.95 | **TP** | match | 1/1 | 1/3 |
| #2 | 0.90 | **TP** | match | 2/2 | 2/3 |
| #3 | 0.80 | **FP** | duplicate | 2/3 | 2/3 |
| #4 | 0.70 | **FP** | no truth here | 2/4 | 2/3 |
| #5 | 0.60 | **TP** | match | 3/5 | 3/3 |
| #6 | 0.40 | **FP** | no truth here | 3/6 | 3/3 |

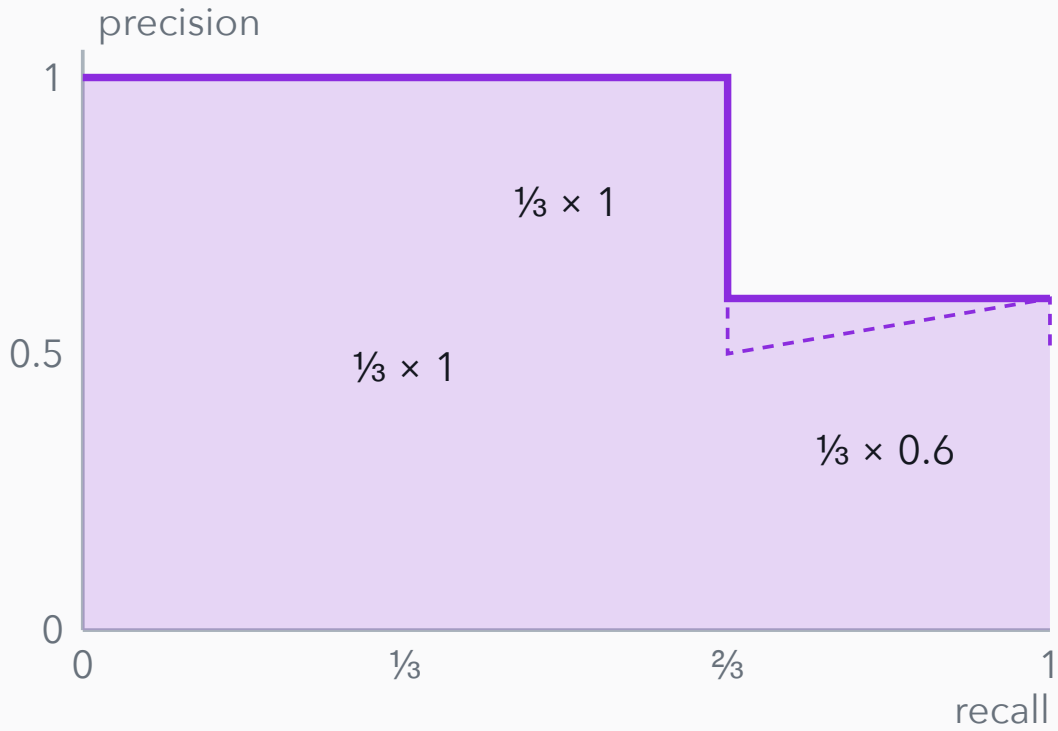
A small invented list (3 objects, 6 detections) so that hits and misses interleave.




> #6 (0.40): FP. 3/6, 3/3. Every point is one possible threshold.

▶ Notebook, part 8

# Average precision: the area under the curve




Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

AP = 1 only if every object is found before the first false positive, as for the street's people.

> AP rewards ranking: correct detections should come before wrong ones. The street people scored AP = 1.

▶ Notebook, part 8

# mAP: average over classes, and over IoU thresholds

$$\mathrm{mAP}_{50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c \quad (\text{a hit needs IoU} \geq 0.5)$$

$$\mathrm{AP}_{[.5:.95]} = \text{mean of mAP at IoU } 0.50, 0.55, \ldots, 0.95$$

| our 8 photos, 7 classes, 31 objects | mAP@0.5 | AP@[.5:.95] | COCO val (published) |
|---|---|---|---|
| YOLOv8n | 1.000 | 0.936 | 37.3 |
| DETR (ResNet-50) | 1.000 | 0.882 | 42.0 |

Eight clean photos are not a benchmark: on COCO's 5,000 validation photos the order flips.

> Both detectors on our eight photos: perfect at IoU 0.5; at strict IoU, YOLOv8n 0.94, DETR 0.88. On COCO's validation set the published numbers are 37.3 and 42.0.

Our evaluator on data/annotations.json; COCO numbers: Ultralytics YOLOv8 documentation and Carion et al. (2020).

▶ Notebook, part 8

# Evaluation in code

```python
from torchmetrics.detection import MeanAveragePrecision
metric = MeanAveragePrecision(iou_type="bbox")
metric.update(preds=[{"boxes": boxes, "scores": scores, "labels": labels}],
              target=[{"boxes": gt_boxes, "labels": gt_labels}])
res = metric.compute()          # res["map"] = AP@[.5:.95], res["map_50"] = AP@0.5
```

> Libraries do the matching, ranking and averaging. The notebook also has a twenty-line version that reproduces the walk-through.

# One question, two answers, three matchings

**Question:** a fixed-size output must describe a set of any size, in any order.

### Dense: by place

- A record at every cell of three grids
- Train cells near each object
- Filter by score, NMS

### DETR: by matching

- 100 learned slots read the image
- Hungarian matching, one slot per object
- Threshold; no NMS

| matching | pairs | when |
|---|---|---|
| assignment | outputs ↔ annotated objects | training |
| NMS | predictions ↔ predictions | test time (dense) |
| evaluation | predictions ↔ annotated objects | measuring |

> And three different matchings that are easy to confuse: assignment in training, NMS between predictions, evaluation against annotations. Each uses IoU.

# Next: beyond boxes



A box is a rough answer. Next lecture: which pixels belong to each object (masks), keypoints, depth, and boxes found from words: “the dog on the right”.

Beyond Boxes · Language-grounded vision

Practice: the notebook runs every number in this lecture.

> Boxes are the coarsest description of where an object is. Next: masks, keypoints, and finding objects named in words.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py).

# YOLOv1 (2016): one responsible cell per object

$$7 \times 7 \text{ cells} \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Each cell: two boxes, each (x, y, w, h, confidence), and one set of 20 class probabilities (PASCAL VOC). Only the cell containing an object's centre is responsible for it; x, y are offsets inside that cell, w, h fractions of the image.

Confidence was trained toward the IoU between the box and the truth. A cell could hold at most one object's class.

> Why did YOLOv1 struggle with flocks of birds?

# Anchors: predict offsets from template boxes

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = p_w\, e^{t_w}, \quad b_h = p_h\, e^{t_h}$$

Faster R-CNN, SSD and YOLOv2–v5 place several template boxes (anchors) of chosen sizes and shapes at every cell, and predict small corrections to each. Positives are anchors with high IoU to an object.

Anchor-free detectors (FCOS, YOLOv8) predict distances from the cell centre instead, as in Part 4.

> What must be chosen by hand for an anchor-based detector?

# Two-stage detectors: propose, then classify

**Stage 1**, a region proposal network: about 2,000 boxes that might contain something.
**Stage 2**: crop features for each proposal (RoI pooling), classify it and refine its box.

R-CNN (2014) ran a CNN per proposal; Fast R-CNN (2015) shared one CNN pass; Faster R-CNN (2015) learned the proposals too. Mask R-CNN adds a mask per proposal: Lecture 23.

torchvision's fasterrcnn_resnet50_fpn returns post-processed detections: its NMS is already inside.

> What does the first stage output?

# Focal loss: quiet the easy background

$$\mathrm{FL}(p_t) = -(1-p_t)^{\gamma}\log p_t, \qquad p_t = \begin{cases} p & y=1 \\ 1-p & y=0 \end{cases}$$

A background cell with p = 0.001: $p_t = 0.999$, so with $\gamma = 2$ its loss is multiplied by $(1 - p_t)^2 \approx 10^{-6}$. Easy examples nearly vanish; hard ones keep their weight.

RetinaNet (Lin et al., 2017) made one-stage detectors as accurate as two-stage ones this way.

> Which examples does focal loss down-weight?

# Box formats in practice

| where | format | the dog |
|---|---|---|
| torchvision | corners, pixels | (407, 83, 768, 588) |
| COCO json | (x, y, w, h), top-left, pixels | (407, 83, 361, 505) |
| YOLO label file | normalised ($c_x$, $c_y$, w, h) | (0.612, 0.524, 0.376, 0.789) |
| DETR output | normalised ($c_x$, $c_y$, w, h) | same as YOLO |

> Why do format bugs often go unnoticed?

# Resizing and padding move the boxes too

Detectors often resize a photo to a fixed size and pad the rest (letterboxing). The boxes must get the same treatment: scale every coordinate by the resize factor, then add the padding offset.

$$960 \times 640 \;\rightarrow\; 640 \times 427 \;(r = \tfrac{2}{3}),\ \text{pad } 106 \text{ px above}: \qquad y' = \tfrac{2}{3}\, y + 106$$

Our photos are multiples of 32 on both sides, so this lecture ran the models without padding.

> What happens to the dog's $y_1 = 83$ after this letterbox?

# After DETR

- **Deformable DETR** (2021): each query attends to a few sampled points per feature level; trains about 10× faster.
- **DINO, RT-DETR** (2022–23): better queries and matching; real-time DETRs.
- **YOLOv10** (2024): one-to-many and one-to-one heads; NMS-free YOLO.
- **Open vocabulary** (OWL-ViT, Grounding DINO): classes given as text, matched with CLIP-like embeddings (the language-grounded vision lecture).

> What did DETR's successors mainly improve?